

Platform independent HPC Recursive Bayesian inference Tomographic ionosphere calibration & New perspectives

13 November, 2020
Joshua Albert



Universiteit
Leiden
Leiden Observatory



LOFAR

Topics covered

1. Platform independent HPC with JAX and XLA
2. Radio interferometry and ionospheric calibration
3. Recursive Bayesian estimation
4. A global-community “Radio Interferometry Kaggle”

High performance, composable, computing on CPU, GPU, & TPU



The problem:

1. Programming on CPU, GPU, and TPU require different (overlapping) skill sets
 - a. GPU SDK's, C++, interfacing to python, scheduling the computation graph, memory transfer, debug many languages, etc.

2. Premature optimisation:

“The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.” *Donald Knuth, TAOcp*

3. Bottlenecks in collaboratively getting new ideas into production.

What is JAX?



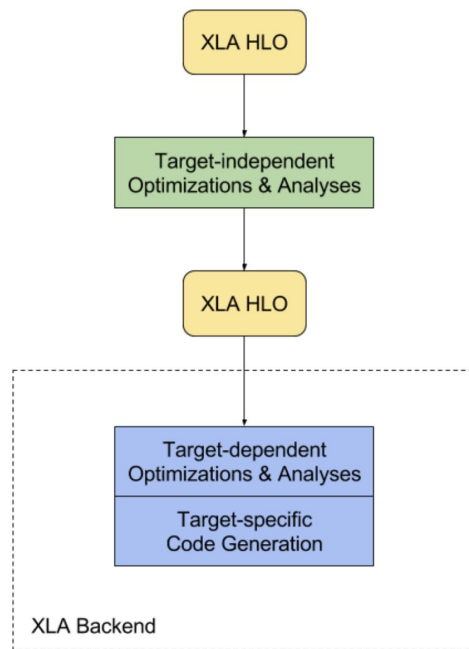
JAX is a new research-focused framework by Google that solves these problems!

- Just-in-time (JIT) compile composable functions into XLA-optimized kernels
- Single-program multiple-data (SPMD) programming over distributed accelerators
- Vectorisation
- Auto-differentiation
- Familiar if you've used numpy or scipy
- Fast!

What is XLA?

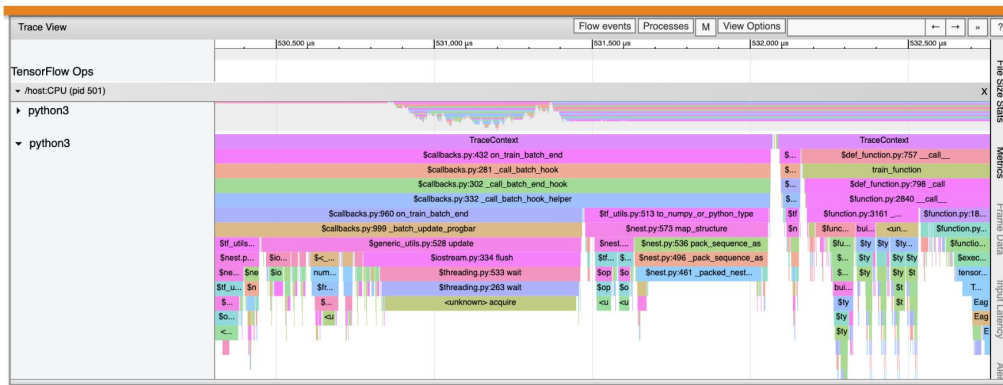
XLA was built to:

- *Improve execution speed.*
 - a. CSE, operation fusing, memory buffer optimisation
- *Improve memory usage.*
 - a. Analyze and schedule memory usage, eliminating many intermediate storage buffers.
- *Reduce reliance on custom Ops.*
 - a. Matches the performance of custom Ops that were fused by hand.
- *Improve portability.*
 - a. Make it relatively easy to write a new backend for novel hardware

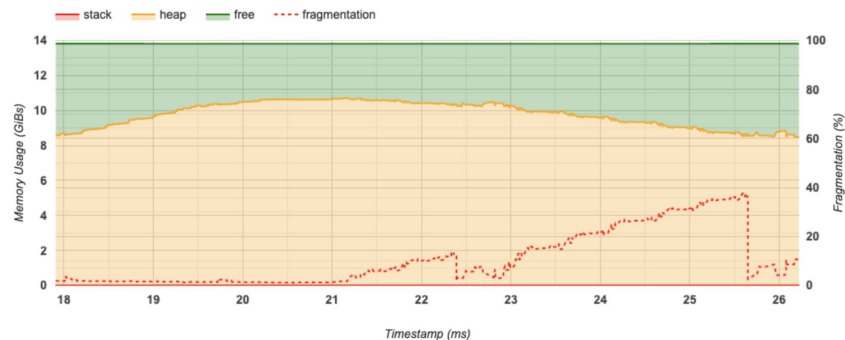


How does JAX work?

- Platform independent description of an algorithm.
- Code tracing and compilation with XLA
- Smartly scheduled asynchronous dispatching to a backend



Smart scheduling the computation graph at runtime



Reduces memory fragmentation

JAX is simple and powerful

- Provides an identical API, and most ops from numpy and scipy.

```
from jax import numpy as jnp
from jax.scipy.special import ndtri
```

- New ideas can be *quickly* implemented in a *platform independent* way.

```
import jax.numpy as jnp
from jax import jit
```

```
@jit
def select_tril(x):
    mask = jnp.arange(x.shape[0])[:, None] > jnp.arange(x.shape[1])
    return jnp.where(mask, x, jnp.zeros_like(x))
```

```
import numpy as np
x = np.arange(12).reshape((3, 4)) # mix in numpy as desired
select_tril(x)
```



High-level optimisation (HLO)
representation (before XLA compile)

```
{ lambda a b ; c.
  let d = xla_call[ backend=None
    call_jaxpr={ lambda ; a b c.
      let d = broadcast_in_dim[ broadcast_dimensions=(0,)
        shape=(3, 1) ] a
        e = broadcast_in_dim[ broadcast_dimensions=(1,)
        shape=(1, 4) ] b
        f = gt d e
        g = broadcast_in_dim[ broadcast_dimensions=( )
        shape=(3, 4) ] 0
        h = xla_call[ backend=None
          call_jaxpr={ lambda ; a b c.
            let d = select a b c
              in (d,) }
          device=None
          donated_invars=(False, False, False)
          name=_where ] f c g
        in (h,) }
      device=None
      donated_invars=(False, False, False)
      name=select_tril ] a b c
    in (d,) }
```

XLA vs hand-tuned code



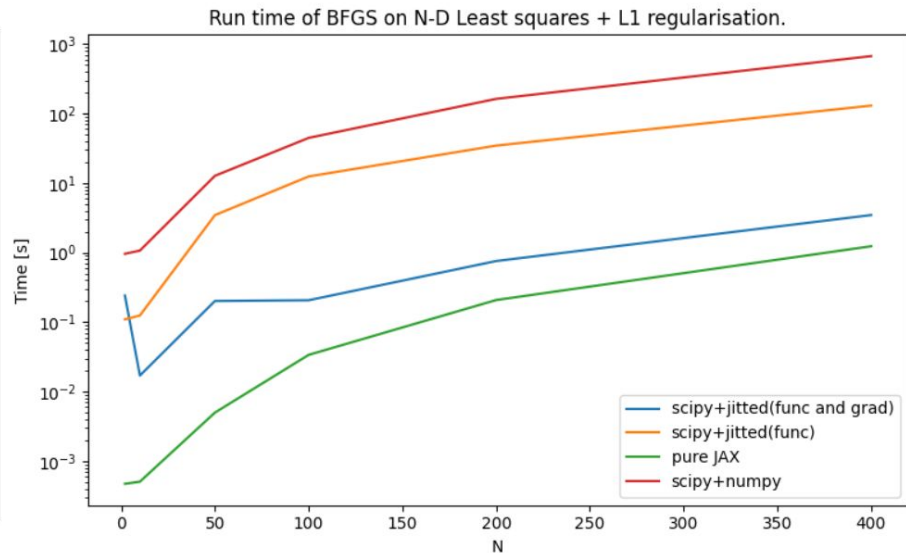
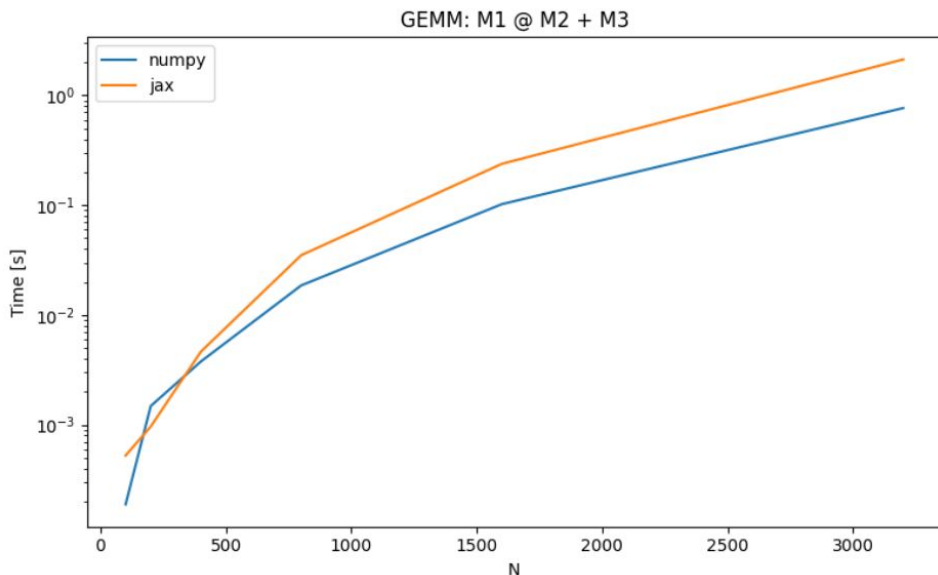
No general answer:

- Are you using CPU/GPU/TPU?
 - TPUs only work with XLA.
- Currently, the compiler team at Google has been given DL kernels as input.
 - They put development effort into directions based on demand of use case.
 - Opportunity to send a researcher to liaise on RCI-specific XLA optimisations.
- Is the hand-tuned code many strung-together individually optimised kernels?
 - XLA almost always faster in this case.
- Typically people know methods or computer science *but rarely both*.

In most tests, XLA is just as fast as expertly tuned code. But, to be sure you need to profile on the correct hardware.

How fast is JAX?

- Same speed on simple highly optimised linear algebra like **GEMM**
- Much faster when whole algorithms are compiled and operations + metadata can be fused





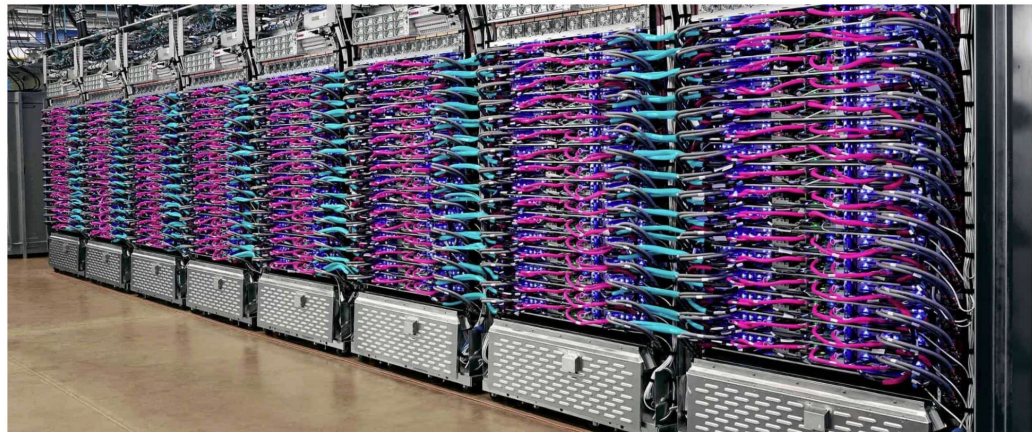
How fast is JAX?

Question: can we get 5 orders speed up too?

News: JAX tops largest-scale MLPerf Training 0.7 benchmarks!

In 2020, JAX does in under 30 seconds with TPUs what took 3 weeks in 2015 on the top hardware.
⇒ 5 orders of magnitude speed up in 5 years! 430 PFLOPs achieved!

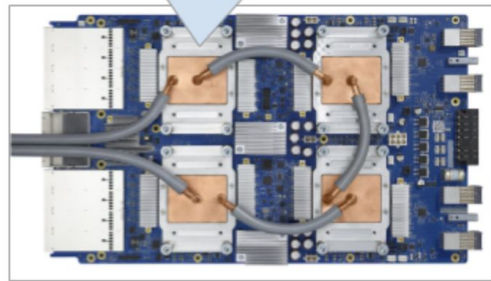
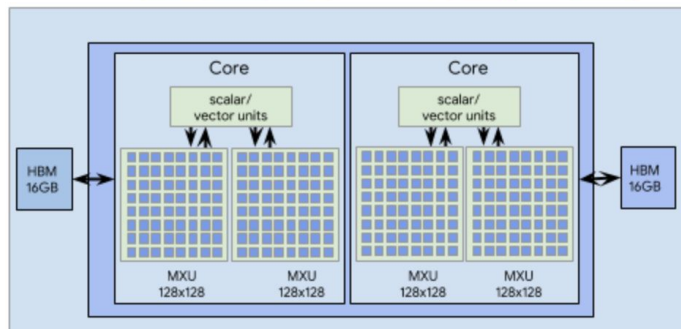
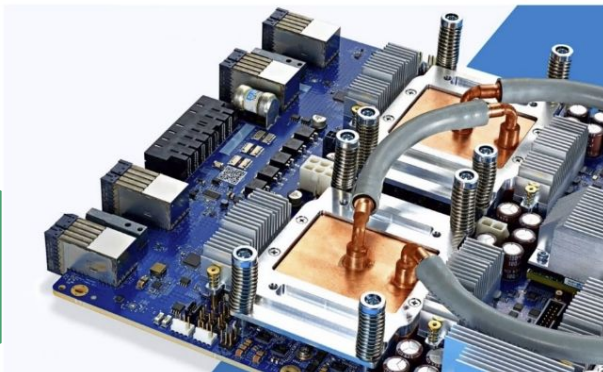
Benchmark	Framework	Number of accelerators	Time to convergence (seconds)
ResNet-50	TensorFlow	4096	29 sec
	JAX	4096	28 sec
BERT	TensorFlow	4096	23 sec
	JAX	4096	24 sec
SSD	TensorFlow	4096	28 sec
	JAX	2048	33 sec
Transformer	TensorFlow/ Lingvo	4096	21 sec
	JAX	4096	16 sec



What is a TPU?

- 50cm long ASIC boards with 4 chips/8 cores.
 - 4 traditional vector processing units (VPU)
 - 4 dedicated matrix multiplication (MXU) capable of processing 128×128 matrices
- They are so fast that feeding them data becomes a bottleneck.
- Pipelines need to adopt IO strategies to keep the TPUs busy.
- JAX can handle scheduling of dispatch.
- Typically, you need asynchronous file reading operations with $\sim 100\text{MB}$ chunking.

See John Romain's talk for more about TPU's



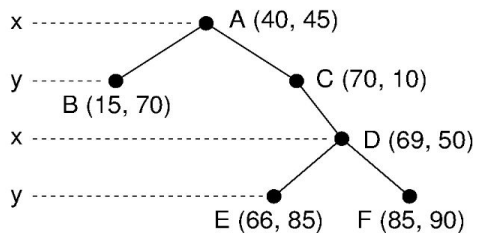
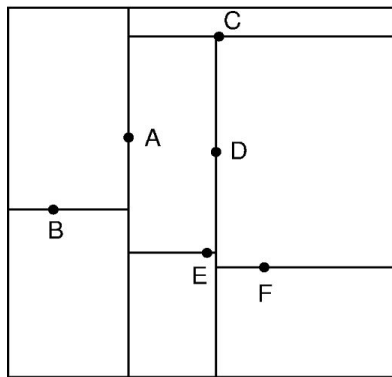
TPU v3 - 4 chips, 2 cores per chip

Thinking in JAX

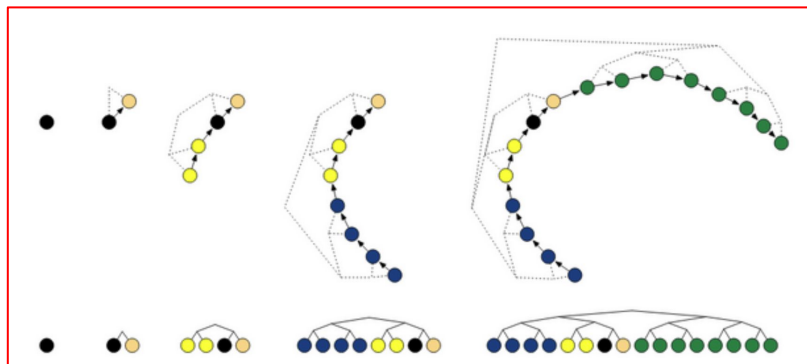
- Memory allocations and shapes must be static at compile time.

You must rework recursive algorithms into iterative algorithms, and forget about dynamic mallocs.

Standard implementations of algorithms like KD-Trees are not possible.



However, typically it's always possible to transform a dynamic algorithm to a static one.



Notes on JAX



1. JAX works with dask (dask solves a different problem).
2. JAX is not numba.
 - a. Numba vectorises ops over broadcastable dimensions
 - b. JAX pushes vectorised dimensions down to the lowest level.
3. JAX is great for SPMD programs with sharded inputs.
4. Auto-differentiation - A very convenient tool for quickly building optimisation algorithms.
5. Extracting XLA programs for accelerator streaming is possible *in principle*.
 - a. Community use case driven.

JAX-based Nested Sampling

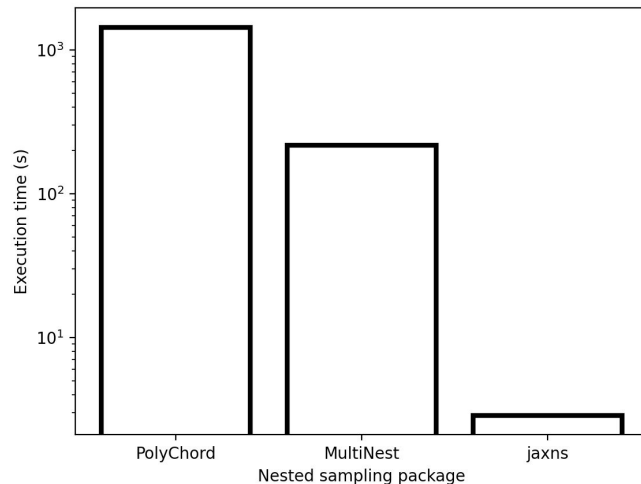
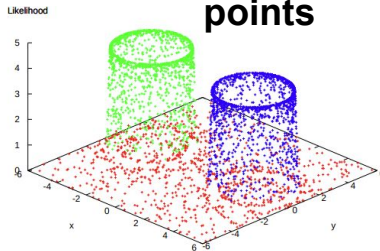
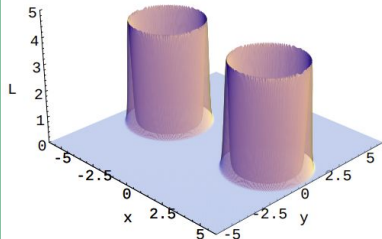
Traditionally a slow algorithm $\Rightarrow$ only used for offline inference.

By writing the entire algorithm in JAX and letting XLA optimise it, we get orders of magnitude improvements.



Toy model inference problem

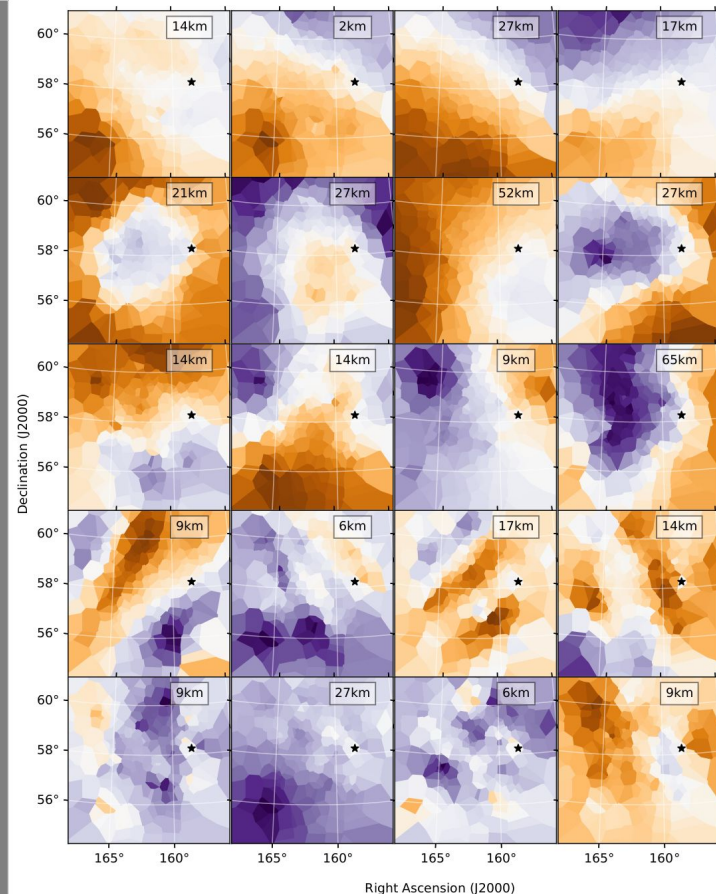
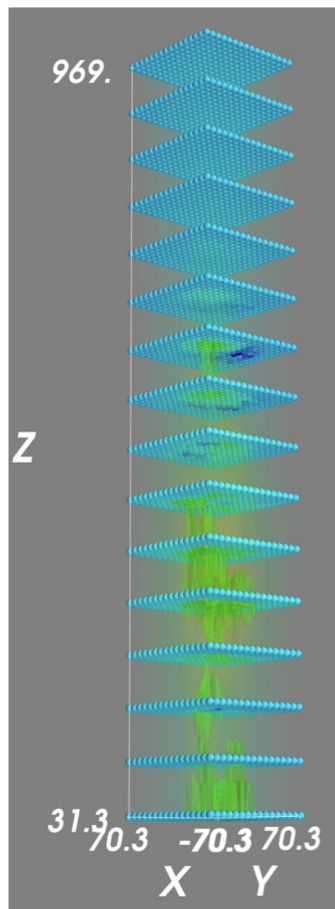
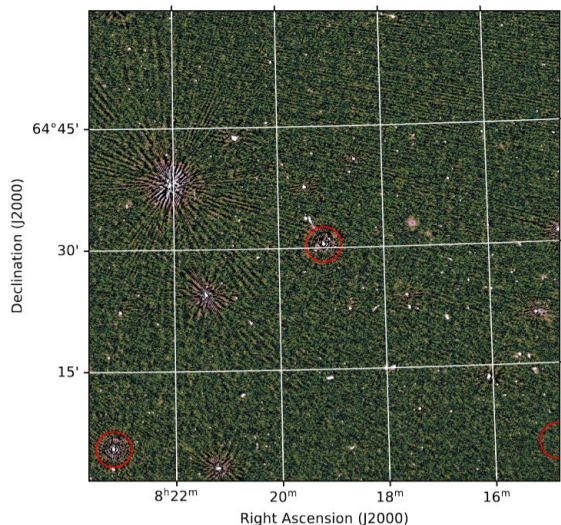
**30-dim
1000 live
points**



[1] <https://github.com/Joshuaalbert/jaxns>

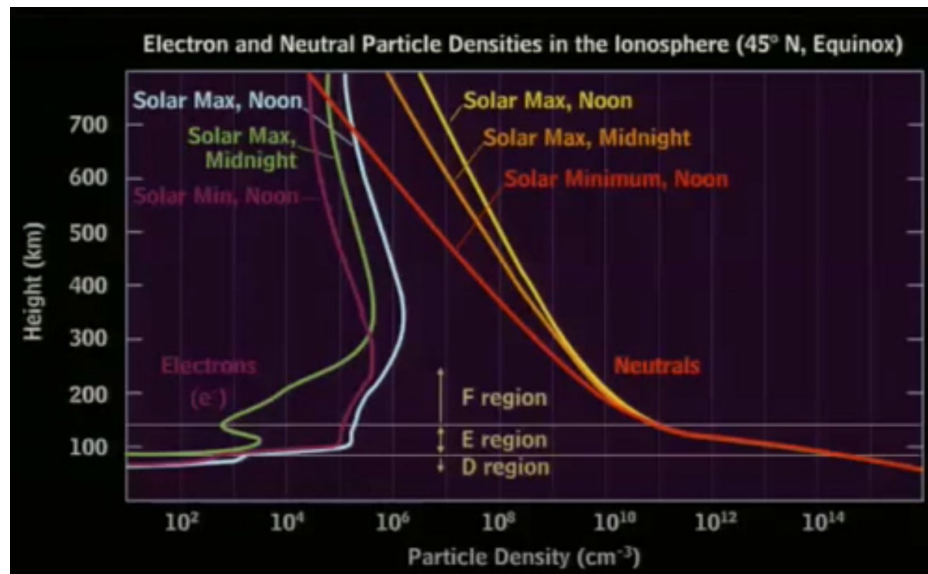
Ionosphere calibration

Can we use the physics of
electromagnetism, fluid
dynamics, and probability theory
to calibrate the ionosphere?



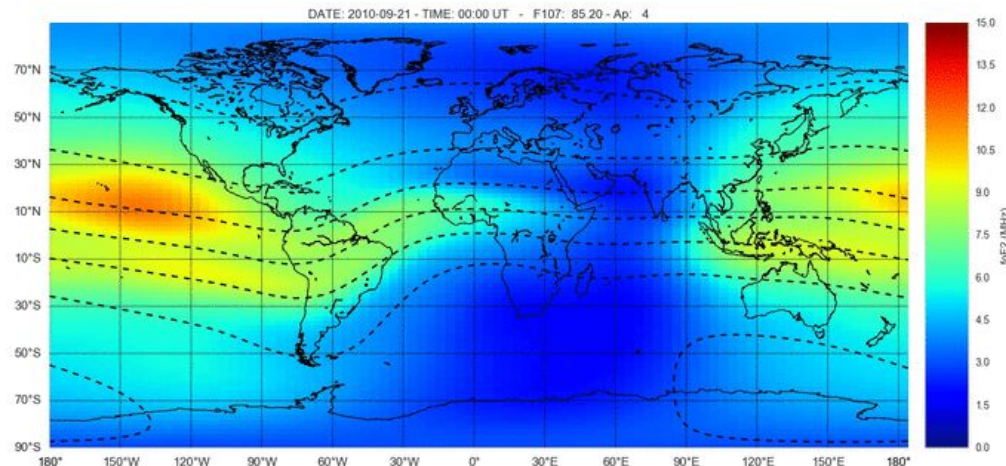
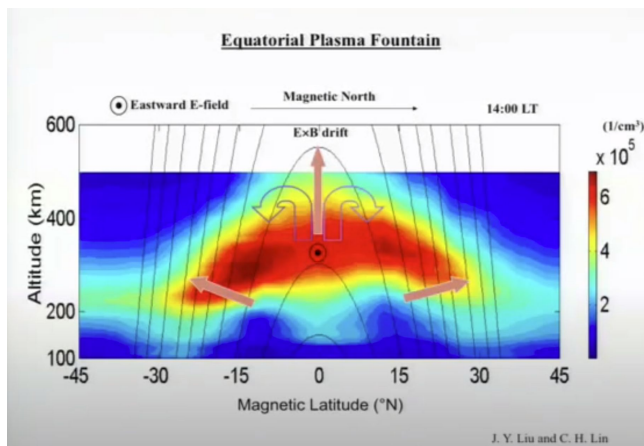
Ionosphere quick review

- E region: 90-150km
 - neutral-ion collisions frequent
- F-region: >150km
 - collisions less frequent
 - ions frozen to the magnetic field
 - Affects how plasma and neutral motions interact.
- Plasma is harder to blow away because it is fixed to mag-field.
- Transfers momentum between altitudes to neutral particles.



Ionosphere quick review

- E-region ion lifetime $\sim$ few minutes
 - $\Rightarrow$ density shrinks at night
 - Persisted (apparently) by coronal scattering of sunlight, and starlight
- F-region ion lifetime $\sim$ hours
 - $\Rightarrow$ plasma persists into the night.



Ionosphere quick review

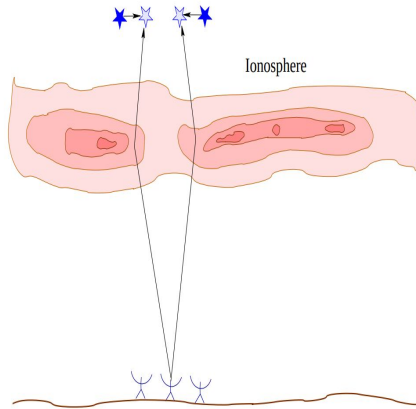
- What drives plasma convection in the evening ionosphere?
- Why is there a strong upward plasma drift shortly after sunset?
- How does convection vary with solar activity?
- How does night-time E-region affect F-region convection?

Check the work of A. Richmond, T.-W. Fang, and A. Matue to understand such questions.

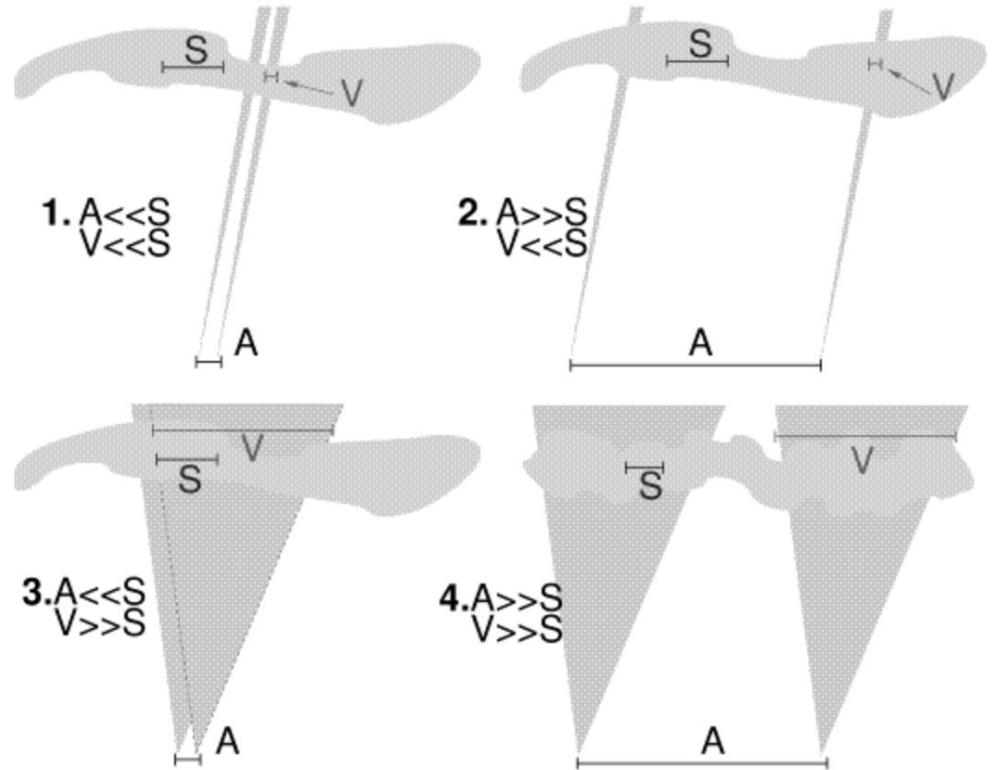
Array configurations

Efficient snapshot surveying has filled aperture and wide field of view.

- Lonsdale-3
- Correlations over the field of view and between antennas

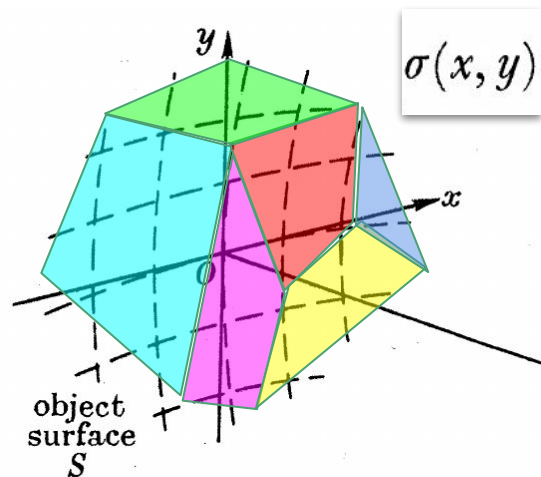


Cotton et al. 2004



Lonsdale 2005

The isoplanatic-patch assumption



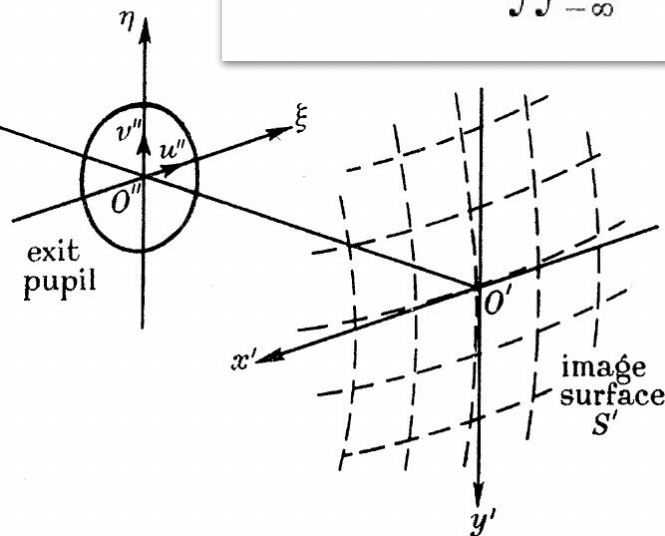
Motivates the idea of facet calibration.

Fellgett and Linfoot 1955

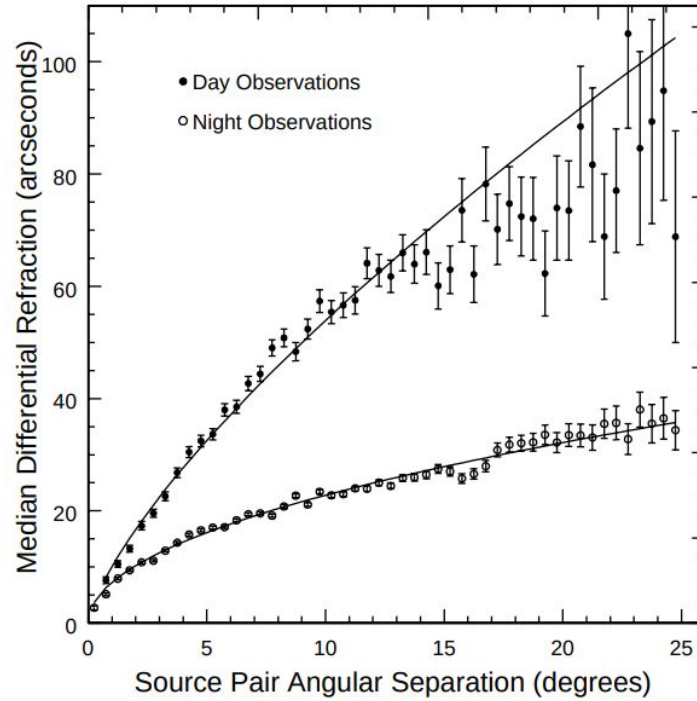
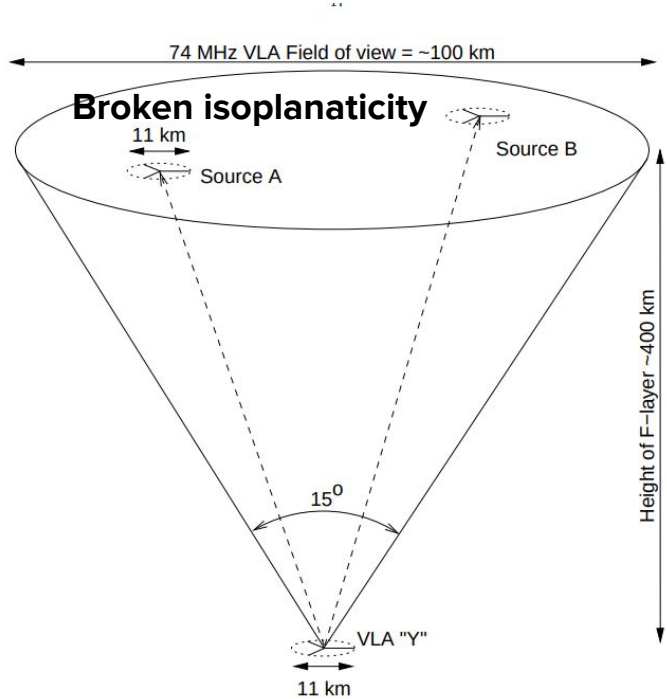
$$w(x', y'; x, y) = w_A(x' - x, y' - y)$$

When this can be represented with a planar convolution

$$I(x', y') = \iint_{-\infty}^{\infty} \sigma(x, y) w(x', y'; x, y) dx dy.$$



A good survey with a wide field low-frequency instrument requires a good ionospheric calibration



Cohen and Röttgering, 2009

LOFAR

LBA: 57MHz
HBA: 150MHZ

Dutch super-terp

Baselines 0.1 - 60km
+ international

Main objective to survey the entire northern sky in unprecedented detail.

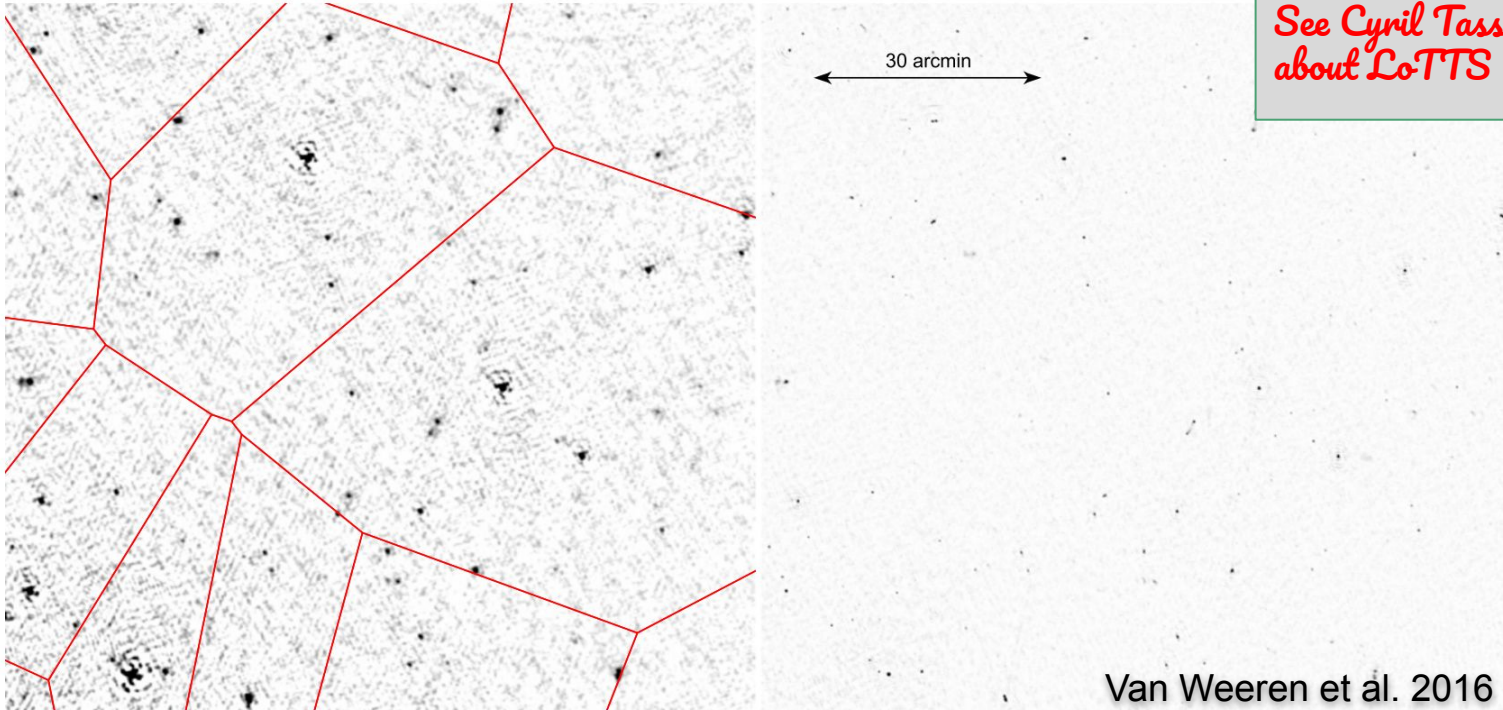
- 2006: start building
- 2012: first light
- 2016: first DD HBA image
- 2019: LoTSS DR1
- 2020?: first DD LBA image, DD 0.3" image

Using 1/100 of data in beam-forming mode (Tb/s) vs. all-sky mode (Pb/s).



See Cyril Tasse's talk for more about LOFAR

In 2016 LOFAR-HBA had its first DD calibrated image, implying the survey mission was close at hand...

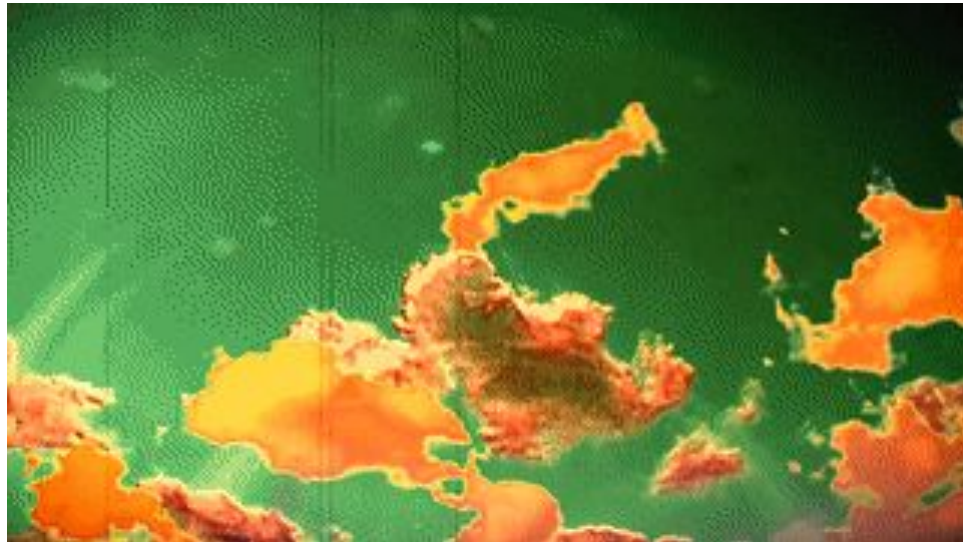


See Cyril Tasse's talk for more about LoTTS

Van Weeren et al. 2016

Ionospheric calibration: overview

1. Review the RIME
2. Introduce the tomographic kernel
3. Remove inter-calibrator ionospheric errors



Radio interferometry and the ionosphere: review

Maxwell's equations for linear dielectrics

$$\nabla \cdot (\epsilon \mathbf{E}) = 0$$

$$\nabla \cdot (\mu \mathbf{H}) = 0$$

$$\nabla \times \mathbf{E} = i\omega \mu \mathbf{H}$$

$$\nabla \times \mathbf{H} = -i\omega \epsilon \mathbf{E}, \quad \Rightarrow$$

$$n = \sqrt{\epsilon_r \mu_r}$$
$$c^{-1} = \sqrt{\epsilon_0 \mu_0}$$

Using vector calculus identities

$$\nabla \cdot (\nabla \times \mathbf{A}) = 0$$

$$\nabla (\nabla \cdot \mathbf{A}) - \nabla \times (\nabla \times \mathbf{A}) = \nabla^2 \mathbf{A}$$

Non-linear wave equation

$$\nabla^2 \mathbf{E} + k^2 \mathbf{E} = \nabla (\nabla \cdot \mathbf{E}).$$

Radio interferometry and the ionosphere: review

Length-scale of changes
in electric permittivity
are bigger than a few
wavelength.

$$k_m^2 r^2 \gg O(25) \Rightarrow \nabla(\nabla \cdot \mathbf{E}) = 0$$

**The inhomogeneous linear wave
equation**

$$(\nabla^2 + k_m^2)E = -k_m^2 \Phi E$$

**Scattering
potential** $\Phi = (n^2 - 1)$

$$n = \sqrt{\epsilon_r \mu_r}$$
$$c^{-1} = \sqrt{\epsilon_0 \mu_0}$$

Radio interferometry and the ionosphere: review

The inhomogeneous wave equation

$$(\nabla^2 + k_m^2)E = -k_m^2 \Phi E,$$

**Helmholtz
Green's
function**

$$\begin{aligned} \tilde{g}(\mathbf{s}) &= \frac{1}{4\pi^2 s^2 - k_m^2} \\ \Rightarrow g(\mathbf{r}, \mathbf{r}') &= \frac{e^{ik_m|\mathbf{r}-\mathbf{r}'|}}{4\pi|\mathbf{r}-\mathbf{r}'|}. \end{aligned}$$

Sommerfeld radiation condition: The sources must be sources, not sinks of energy.

**The general solution for: a linear
dielectric, divergence-free medium**

$$E = k_m^2 \int_V g \Phi E \, dV + \oint_S (g \nabla E - E \nabla g) \cdot d\mathbf{A}$$

The Born approximation

The Born series

$$E_{n+1}^{\text{Born}}(\mathbf{x}) = E_i^{\text{Born}}(\mathbf{x}) + k_m^2 [g \star \Phi E_n^{\text{Born}}](\mathbf{x})$$

From Max Born's
quantum scattering
theory of 1920's

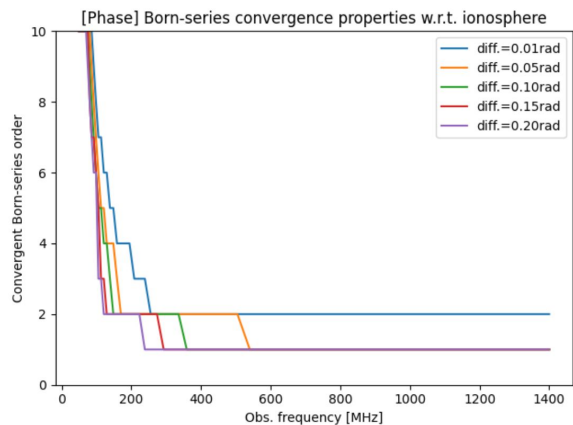
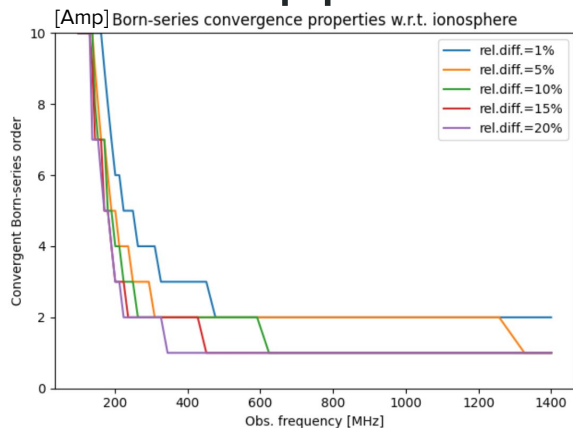
**Initial condition: Incoming
plane wave superposition**

$$E_i^{\text{Born}}(\mathbf{r}) = \int_{\mathbb{S}^2} a(\hat{\mathbf{s}}) e^{ik_m \hat{\mathbf{s}} \cdot \mathbf{r}} d\hat{\mathbf{s}}$$

Convergence criteria: Weak scattering

$$k_m^2 |[g \star \Phi](\mathbf{x})| < 1$$

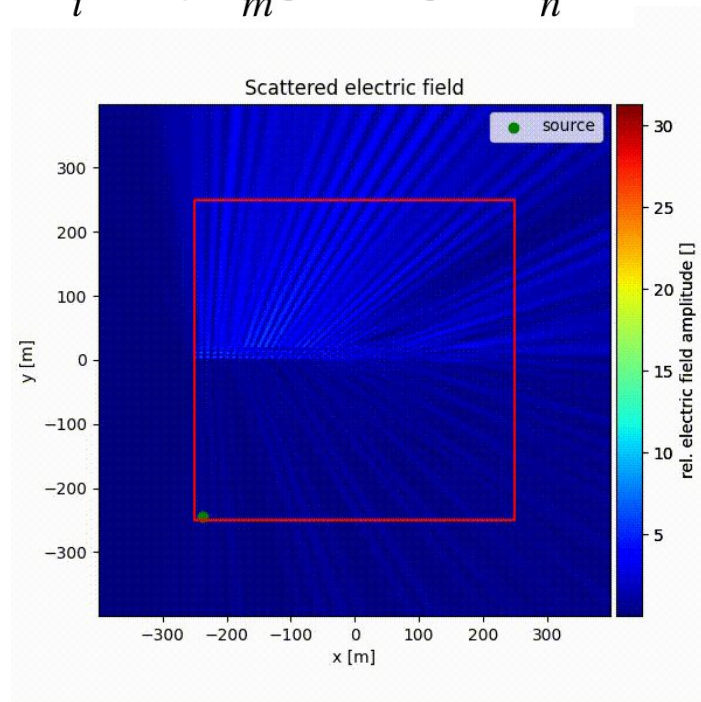
The Born approximation



The first order
Born-approx.
accuracy
increases with
frequency

The Born series

$$E_{n+1}^{\text{Born}} = E_i^{\text{Born}} + k_m^2 \mathcal{F}^{-1} \tilde{G} \mathcal{F} \Phi E_n^{\text{Born}}$$



The Born approximation

The Born RIME

$$V(\mathbf{r}-\mathbf{r}') = \int_{\mathbb{S}^2} e^{ik_m \hat{\mathbf{s}} \cdot (\mathbf{r}-\mathbf{r}')} (\mathcal{J}_i(\hat{\mathbf{s}}) + \mathcal{J}_s(\hat{\mathbf{s}})) d\hat{\mathbf{s}}$$

Linear in both sky and calibration

No gains

Models amplitude and phase calibration simultaneously

Coherence is quadratic in electric field, so is first-order Born relative error.

20% $\Rightarrow$ 4%

5% $\Rightarrow$ 0.25%

Normal sky-image term $\mathcal{J}_i(\hat{\mathbf{s}}) = \langle |a(\hat{\mathbf{s}})|^2 \rangle$

Additive scattered-image term $\mathcal{J}_s(\hat{\mathbf{s}}) = \frac{k_m^2}{4s_z^2} \int_{\mathbb{S}^2} |\tilde{\Phi}(\lambda^{-1}\hat{\mathbf{s}} - \lambda^{-1}\hat{\mathbf{s}}')|^2 \langle |a(\hat{\mathbf{s}}')|^2 \rangle d\hat{\mathbf{s}}'$

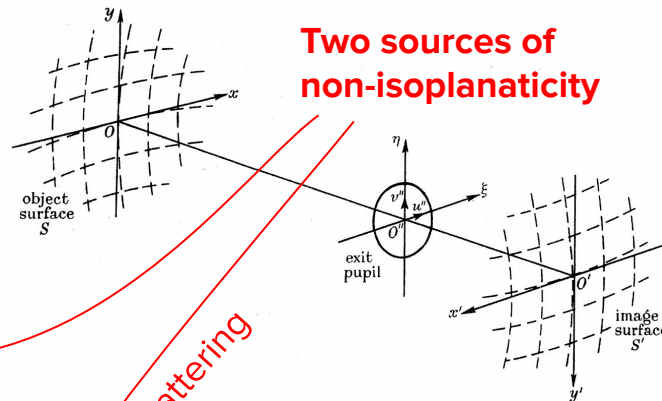
Typically, this convolution kernel is very narrow

The Born approximation

The Born RIME

$$V(\mathbf{r}-\mathbf{r}') = \int_{\mathbb{S}^2} e^{ik_m \hat{\mathbf{s}} \cdot (\mathbf{r}-\mathbf{r}')} (\mathcal{I}_i(\hat{\mathbf{s}}) + \mathcal{I}_s(\hat{\mathbf{s}})) d\hat{\mathbf{s}}$$

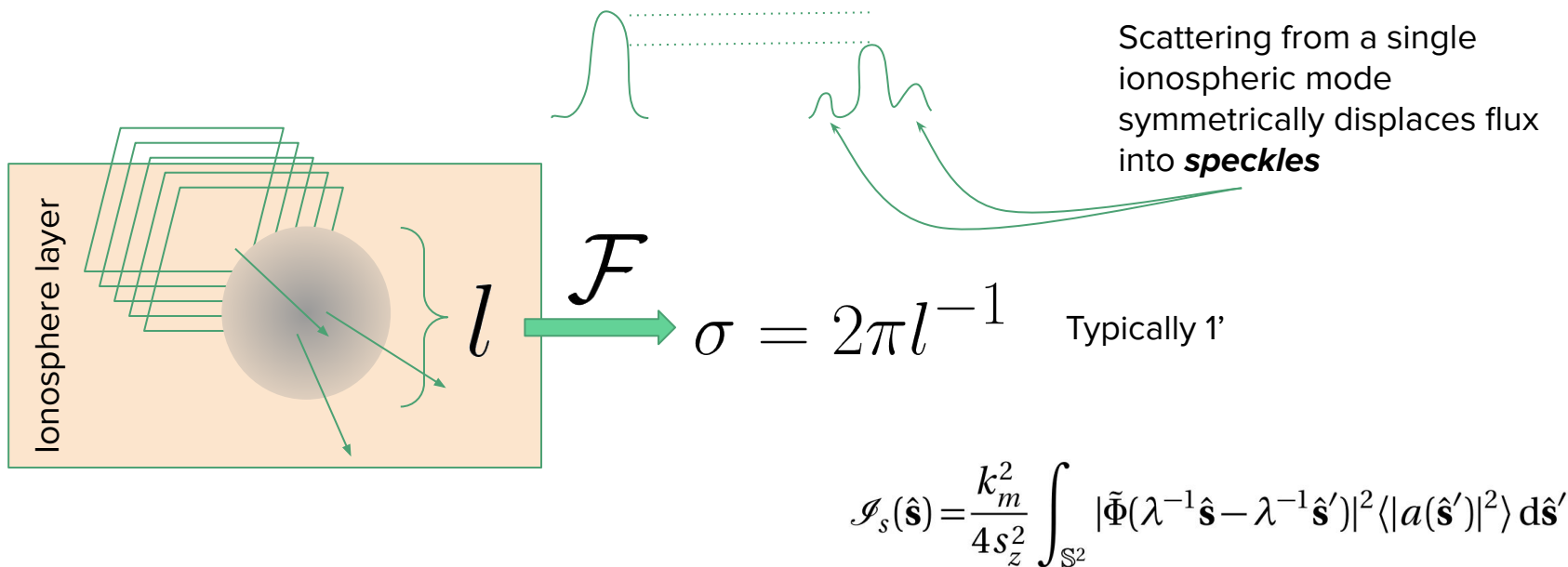
Curvature



Normal sky-image term $\mathcal{I}_i(\hat{\mathbf{s}}) = \langle |a(\hat{\mathbf{s}})|^2 \rangle$

Additive scattered-image term $\mathcal{I}_s(\hat{\mathbf{s}}) = \frac{k_m^2}{4s_z^2} \int_{\mathbb{S}^2} |\tilde{\Phi}(\lambda^{-1}\hat{\mathbf{s}} - \lambda^{-1}\hat{\mathbf{s}}')|^2 \langle |a(\hat{\mathbf{s}}')|^2 \rangle d\hat{\mathbf{s}}'$

The Born Approximation



The Rytov approximation $\Rightarrow$ gains formulation

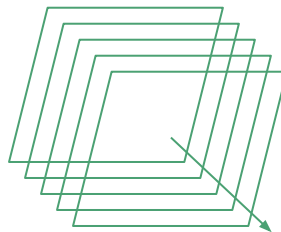
Let the solution be some phase function $\phi(\mathbf{r})$

Eikonal transformation

$$E^{\text{Rytov}}(\mathbf{r}) = E_i^{\text{Rytov}}(\mathbf{r}) e^{\phi(\mathbf{r})}$$

Linearity in phase or “path-length”.

Log-Sum-Exp algebra



The Rytov approximation $\Rightarrow$ gains formulation

The first-order Rytov solution

$$\phi(\mathbf{r}, \hat{\mathbf{s}}) = \frac{k_m^2}{E_i^{\text{Rytov}}(\mathbf{r}, \hat{\mathbf{s}})} [g \star \Phi E_i^{\text{Rytov}}](\mathbf{r}, \hat{\mathbf{s}})$$

$$E_i^{\text{Rytov}} \phi \stackrel{\text{1st ord.}}{=} E_s^{\text{Born}}$$

From Tatarskii, V.
1961 “Wave
Propagation in
Turbulent Medium”

**Initial condition: Eikonal
initial condition**

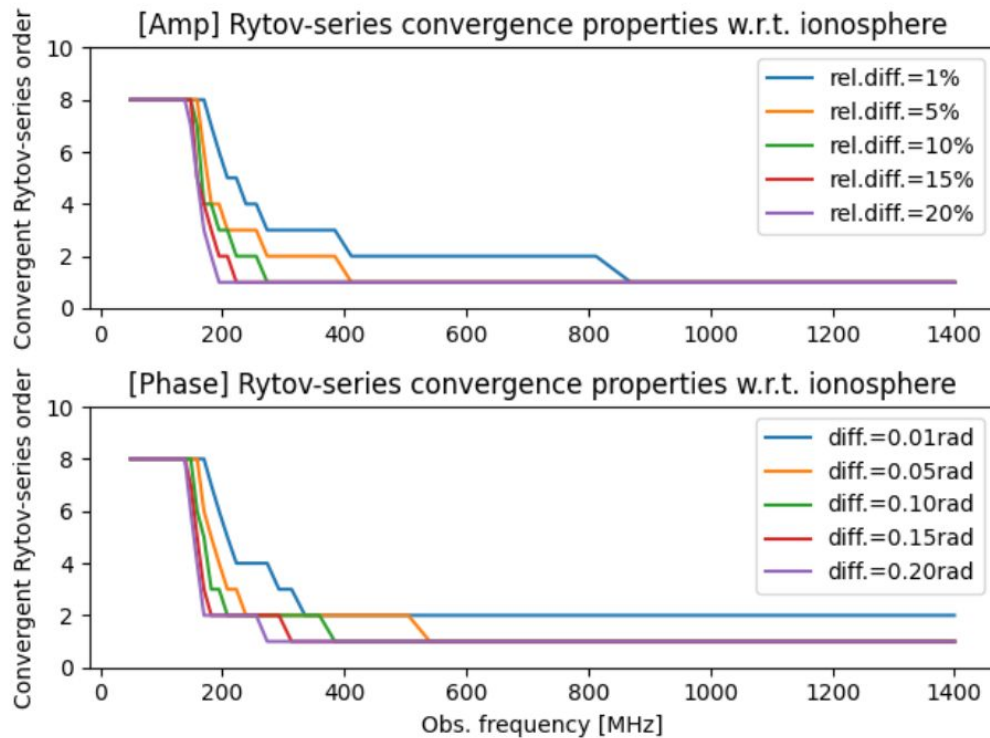
$$(\nabla E_i^{\text{Rytov}})/E_i^{\text{Rytov}} = \nabla \phi_0$$

Convergence criteria: less interpretable

$$|\nabla \phi \cdot \nabla \phi| \ll |k_m^2 \Phi|$$

~~$$\int_{S^2} a(\hat{\mathbf{s}}) e^{ik_m \hat{\mathbf{s}} \cdot \mathbf{r}} d\hat{\mathbf{s}}$$~~

The Rytov approximation $\Rightarrow$ gains formulation



$$E^{\text{Rytov}} = E_i^{\text{Rytov}} e^{\sum_m^n \sigma^m \phi_m}$$

$$E^{\text{Born}} = \sum_m^n \sigma^m E_m^{\text{Born}}$$

Match up same powers in the perturbation parameters

$$\phi_1 = E_1^{\text{Born}} / E_i^{\text{Rytov}}$$

$$\phi_2 = \left(E_2^{\text{Born}} E_i^{\text{Rytov}} - \frac{1}{2} (E_1^{\text{Born}})^2 \right) / E_i^{\text{Rytov}}$$

$$\phi_3 = \left(E_3^{\text{Born}} (E_i^{\text{Rytov}})^2 + \frac{1}{3} (E_1^{\text{Born}})^3 - E_1^{\text{Born}} E_2^{\text{Born}} E_i^{\text{Rytov}} \right) / E_i^{\text{Rytov}}$$

The Rytov approximation $\Rightarrow$ gains formulation

The Rytov RIME

$$V(\mathbf{r}, \mathbf{r}') = \int_{\mathbb{S}^2} G(\hat{\mathbf{s}}, \mathbf{r}) G^*(\hat{\mathbf{s}}, \mathbf{r}') \langle |a(\hat{\mathbf{s}})|^2 \rangle e^{ik_m \hat{\mathbf{s}} \cdot (\mathbf{r} - \mathbf{r}')} d\hat{\mathbf{s}}$$

Negligible diffraction $\Leftrightarrow$ JWKB approximation

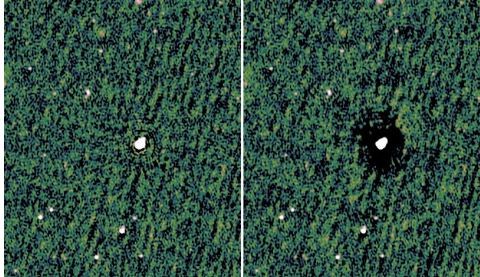
The first-order Rytov phases are the *gains*

All scattering information in gains.

$$G(\hat{\mathbf{s}}, \mathbf{r}) \approx \exp \left[i \frac{k_m}{2} \int_0^\infty \Phi(\mathbf{r} + \eta \hat{\mathbf{s}}) d\eta \right]$$

Phase wrapping non-linearities.

Born RIME vs Rytov RIME



Both are weak-scattering approximations
(typical less than 1% difference)

Born:

- Linear
- Inverse of residual visibilities is the scattered image
- (# ionosphere components) degrees of freedom
- No need to split into facets.

Rytov:

- Non-linear
- Phase wrapping
- (# ant) x (# bright dir) degrees of freedom

The Born RIME

$$\mathcal{I}_i(\hat{\mathbf{s}}) = \langle |a(\hat{\mathbf{s}})|^2 \rangle$$

$$\mathcal{I}_s(\hat{\mathbf{s}}) = \frac{k_m^2}{4s_z^2} \int_{\mathbb{S}^2} |\tilde{\Phi}(\lambda^{-1}\hat{\mathbf{s}} - \lambda^{-1}\hat{\mathbf{s}}')|^2 \langle |a(\hat{\mathbf{s}}')|^2 \rangle d\hat{\mathbf{s}}'$$

$$V(\mathbf{r} - \mathbf{r}') = \int_{\mathbb{S}^2} e^{ik_m \hat{\mathbf{s}} \cdot (\mathbf{r} - \mathbf{r}')} (\mathcal{I}_i(\hat{\mathbf{s}}) + \mathcal{I}_s(\hat{\mathbf{s}})) d\hat{\mathbf{s}}$$

The Rytov RIME

$$V(\mathbf{r}, \mathbf{r}') = \int_{\mathbb{S}^2} G(\hat{\mathbf{s}}, \mathbf{r}) G^*(\hat{\mathbf{s}}, \mathbf{r}') \langle |a(\hat{\mathbf{s}})|^2 \rangle e^{ik_m \hat{\mathbf{s}} \cdot (\mathbf{r} - \mathbf{r}')} d\hat{\mathbf{s}}$$

cf. quantitative comparisons of Born and Rytov (Keller 1969, Slaney et al 1984, Oristaglio 1987, Beydoun and Tarantola 1987)

A physics based model for ionospheric corrections

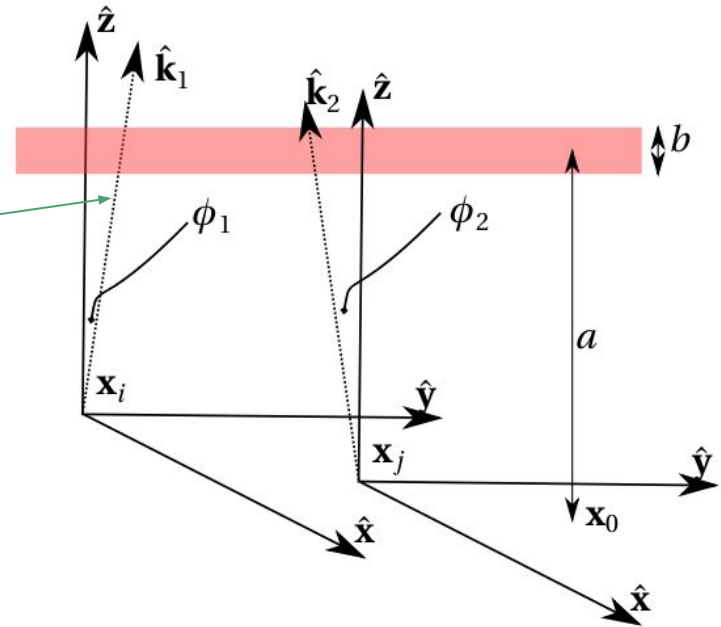
Schematic of the ionosphere

- an infinite in extent, **flat, thick, stochastic** layer at some height above the reference antenna.
- and let the free electron density (FED) be a **Gaussian random field**.

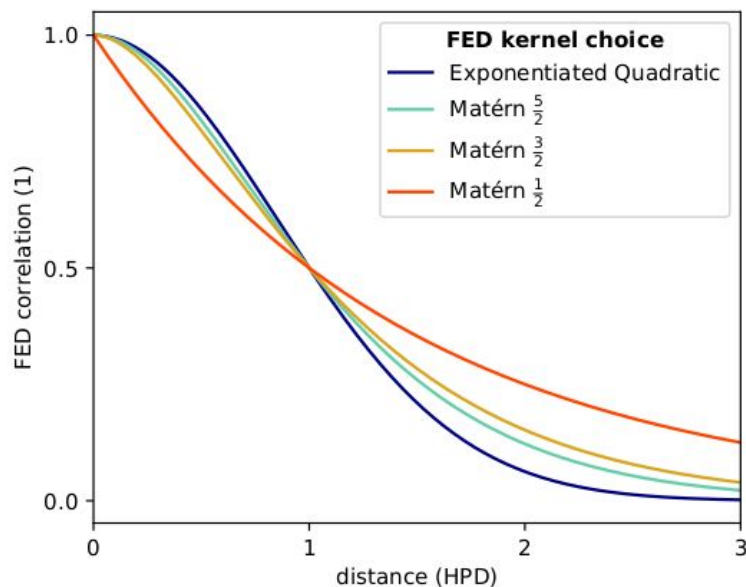
$$G(\hat{\mathbf{s}}, \mathbf{r}) \approx \exp \left[i \frac{k_m}{2} \int_0^\infty \Phi(\mathbf{r} + \eta \hat{\mathbf{s}}) d\eta \right]$$

$$\phi = -8.4470 \frac{TEC}{[\text{mTECU}]} \frac{\nu^{-1}}{[\text{MHz}^{-1}]}$$

Line of sight TEC



Review: Gaussian processes



$$\mu : \mathcal{X} \rightarrow \mathbb{R}$$

Mean function

$$K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$$

Covariance function/kernel

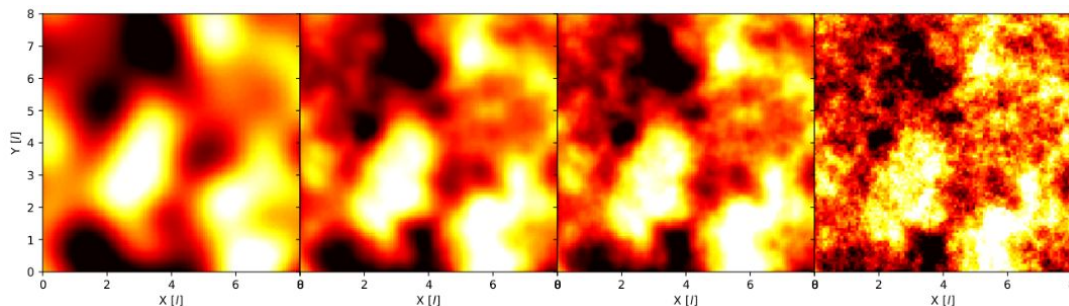
$$n_e \sim \mathcal{N}[\mu, K]$$

Gaussian process

$$\begin{aligned}\mu_i &= \mu(\mathbf{x}_i) \\ \Sigma_{ij} &= K(\mathbf{x}_i, \mathbf{x}_j)\end{aligned}$$

Marginal likelihood

$$p(\mathbf{Y} \mid \mu, K) = \mathcal{N}(\mathbf{Y} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma} + \boldsymbol{\Sigma}_{\text{obs}})$$



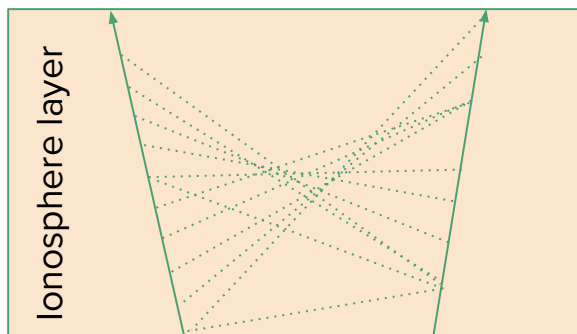
Example realisations with different kernels

Tomographic kernel

What is the covariance function for differential TEC?

$$G(\hat{\mathbf{s}}, \mathbf{r}) \approx \exp \left[i \frac{k_m}{2} \int_0^\infty \Phi(\mathbf{r} + \eta \hat{\mathbf{s}}) d\eta \right]$$

Integrate the FED kernel over all possible lines between pairs of points on



$$\mathcal{X} = \underbrace{\mathbb{R}^3 \times \mathbb{S}^2 \times \mathbb{R}}_{(\text{ant}, \text{dir}, \text{time})}$$

6 dimensional space

$$\mathbf{MX} \sim \mathcal{N}(\mathbf{M}\boldsymbol{\mu}, \mathbf{M}\boldsymbol{\Sigma}\mathbf{M}^T)$$

Linear transformation of a Gaussian is still Gaussian.

$$I(\mathbf{x}_1 - \mathbf{x}_2, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) = \int_{s_1^-}^{s_1^+} \int_{s_2^-}^{s_2^+} k(\mathbf{x}_1 - \mathbf{x}_2 + \hat{\mathbf{n}}_1 t_1 - \hat{\mathbf{n}}_2 t_2) dt_1 dt_2$$

4 (expensive) terms

$$k_{\Delta \text{TEC}}(\mathbf{x}_1, \hat{\mathbf{n}}_1, \mathbf{x}_2, \hat{\mathbf{n}}_2) = I(\mathbf{x}_1 - \mathbf{x}_2, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) - I(\mathbf{x}_0 - \mathbf{x}_2, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) - I(\mathbf{x}_1 - \mathbf{x}_0, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) + I(0, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2)$$

Tomographic kernel

Difficult to work with in the spatial domain due to the double integral,

$$I(\mathbf{x}_1 - \mathbf{x}_2, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) = \int_{s_1^-}^{s_1^+} \int_{s_2^-}^{s_2^+} k(\mathbf{x}_1 - \mathbf{x}_2 + \hat{\mathbf{n}}_1 t_1 - \hat{\mathbf{n}}_2 t_2) dt_1 dt_2$$

Go to Fourier space,

$$S(\mathbf{s}) = \int k(\mathbf{r}) e^{-2\pi i \mathbf{s} \cdot \mathbf{r}} d\mathbf{r}$$

and carry out the ray integrals,

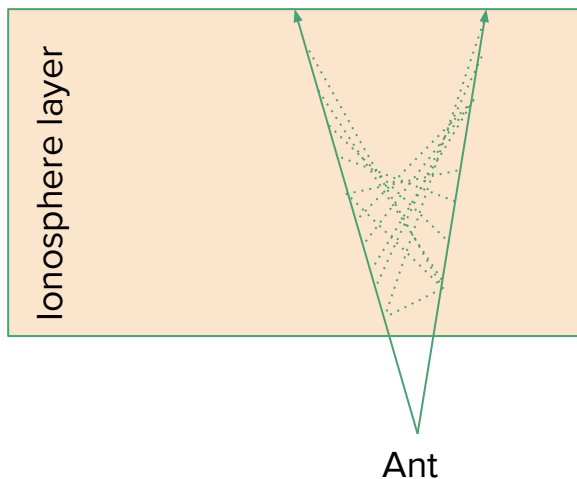
$$I(\mathbf{x}_1 - \mathbf{x}_2, \hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2) = \mathcal{F}^{-1} \left[S(\mathbf{s}) \frac{(e^{2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_1 s_1^+} - e^{2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_1 s_1^-})}{2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_1} \frac{(e^{-2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_2 s_2^+} - e^{-2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_2 s_2^-})}{-2\pi i \mathbf{s} \cdot \hat{\mathbf{n}}_2} \right] (\mathbf{x}_1 - \mathbf{x}_2)$$

Now we can use FFT's and/or make analytical approximations!

Tomographic kernel

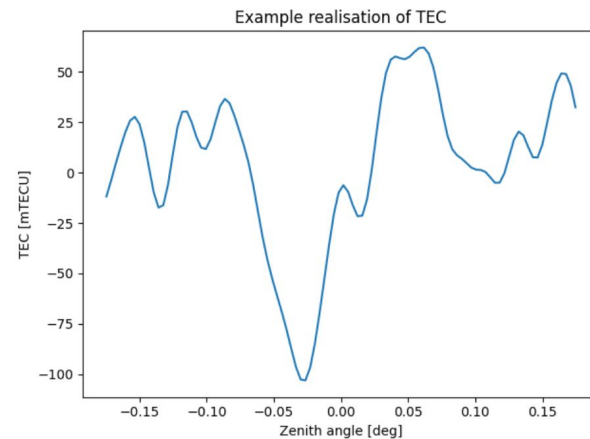
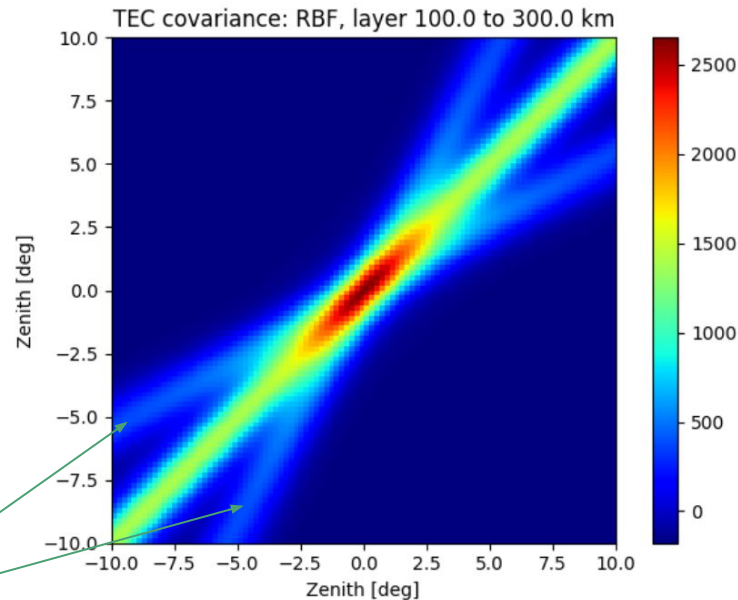
$$\mathcal{X} = \underbrace{\mathbb{R}^3 \times \mathbb{S}^2 \times \mathbb{R}}_{(\text{ant}, \text{dir}, \text{time})}$$

We can ignore
inter-antenna correlations
to reduce comp. complexity



Encodes geo-spectral
properties of
ionosphere.

It's non-stationary
over direction.

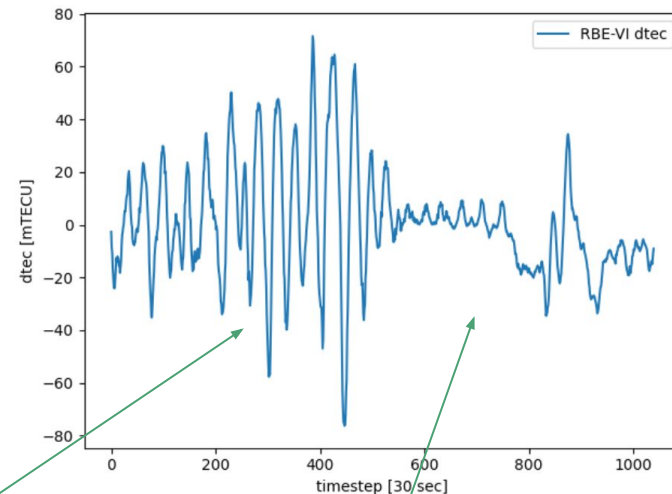


Temporal aspect

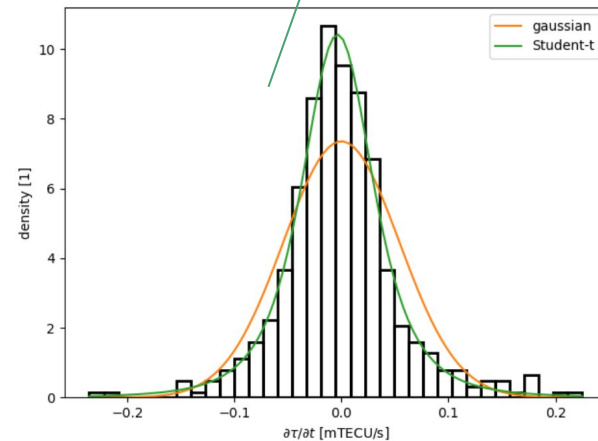
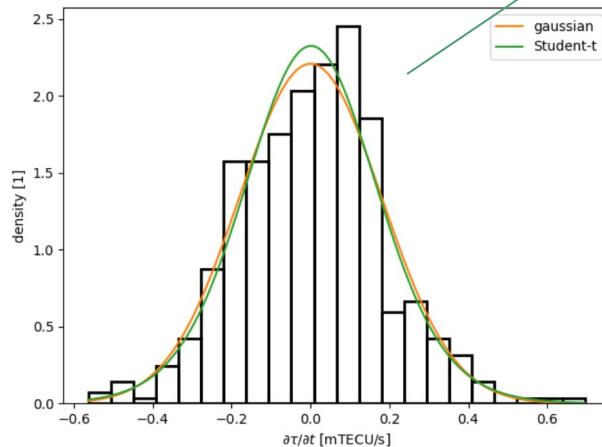
Temporal statistics change over hours.

This allows us to test the Gaussian hypothesis.

This is motivation to go to Student-T processes.



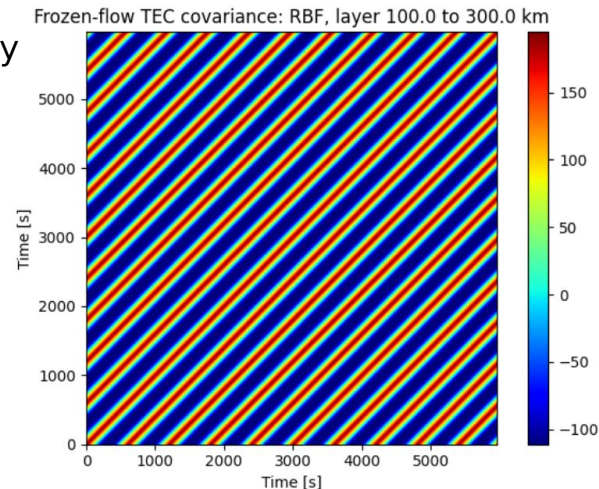
$$\mathbf{MX} \sim \mathcal{N}(\mathbf{M}\boldsymbol{\mu}, \mathbf{M}\boldsymbol{\Sigma}\mathbf{M}^T)$$



Frozen flow assumption

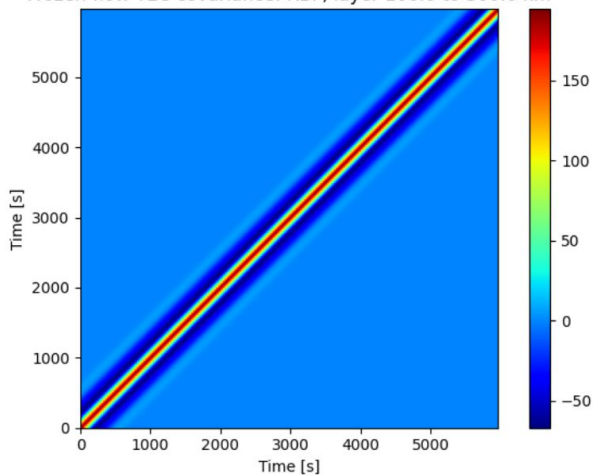
The same structure flows over array with a fixed velocity, $\Delta \mathbf{x} = \mathbf{x}(t_2) - \mathbf{x}(t_1) = \mathbf{v}(t_1 - t_2)$

Perfectly frozen

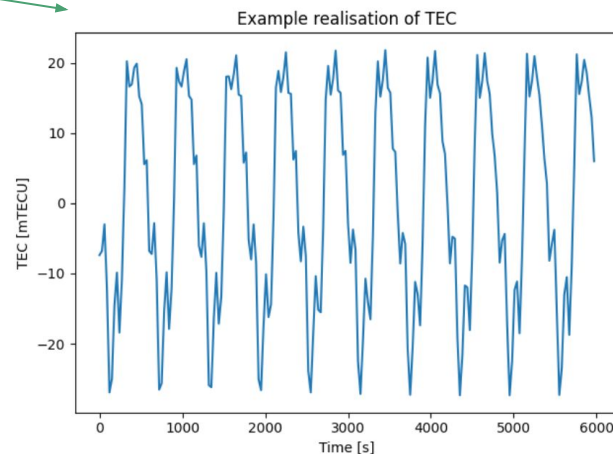
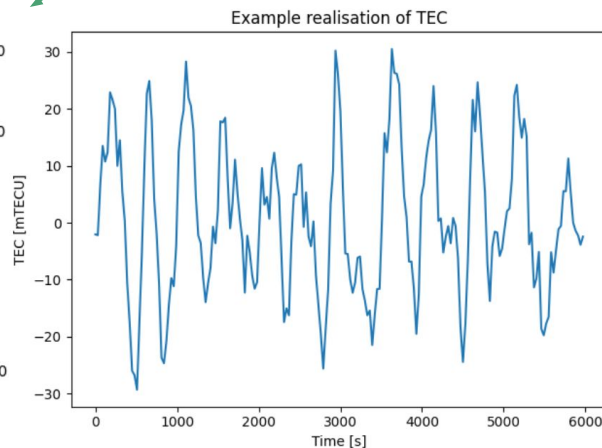


Flow changes over 30 min

Frozen-flow TEC covariance: RBF, layer 100.0 to 300.0 km



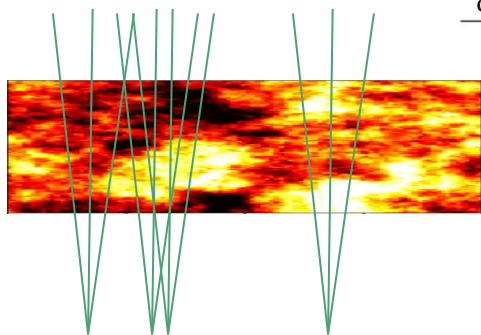
200m/s flow velocity



Exploring the tomographic kernel

Can we achieve super-resolution phase-screen inference?

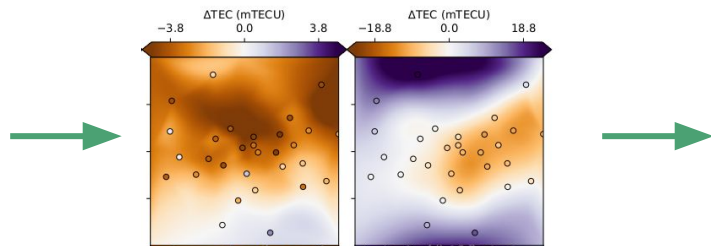
Step 1) simulate
ionosphere and TEC



Core
antennas

Remote
antennas

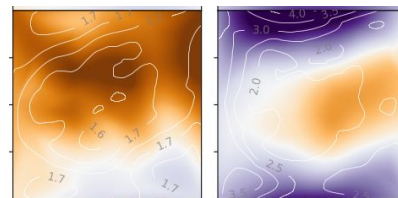
Variety	a (km)	b (km)	K_{FED}	σ_{n_e} (m^{-3})	HPD (km)
dawn	250	100	M32	$6 \cdot 10^9$	15
dusk	350	200	EQ	$3 \cdot 10^9$	15



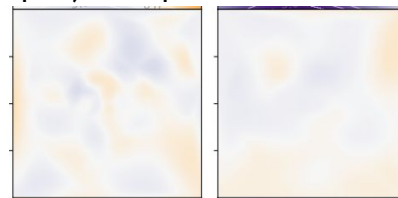
Core
antenna
obs. + truth

Remote
antenna
obs. + truth

Step 2) inference with
tomographic kernel

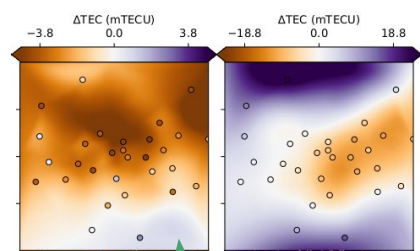


Step 3) compare with truth



Exploring the tomographic kernel

How does that compare to a stationary Gaussian process?

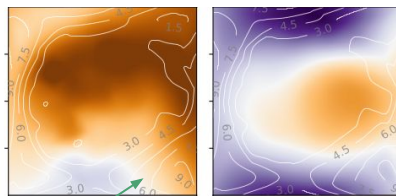


Core antenna obs. + truth

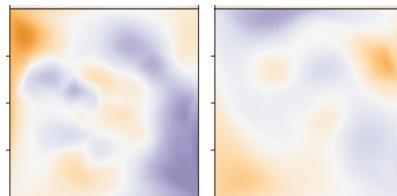
Remote antenna obs. + truth

Region without nearby gain

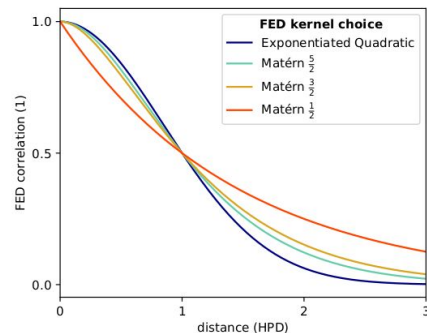
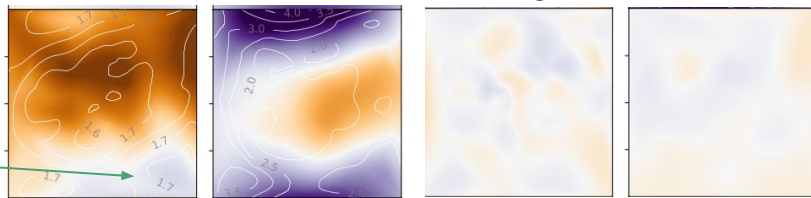
Best of all competitors



Residuals

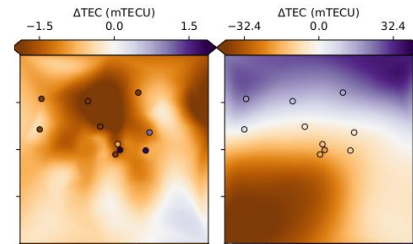


Compare with tomographic



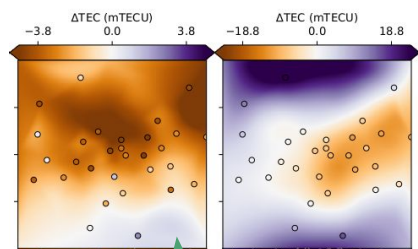
Competitor GP models

Also works in sparse+noisy situations like this



Exploring the tomographic kernel

How does that compare to a stationary Gaussian process?

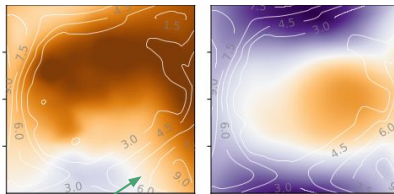


Core antenna obs. + truth

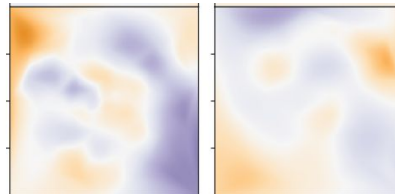
Remote antenna obs. + truth

Region without nearby gain

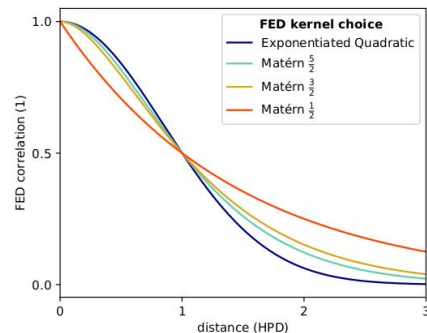
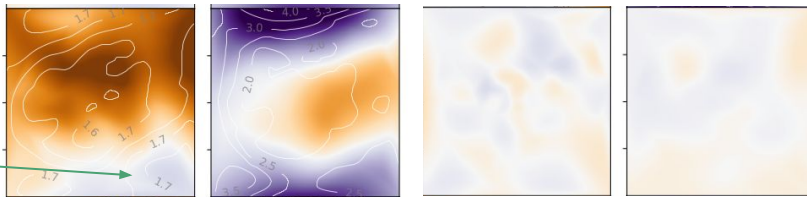
Best of all competitors



Residuals

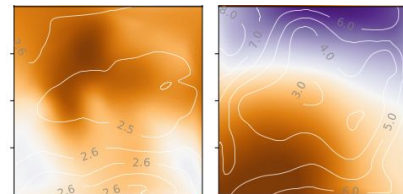


Compare with tomographic



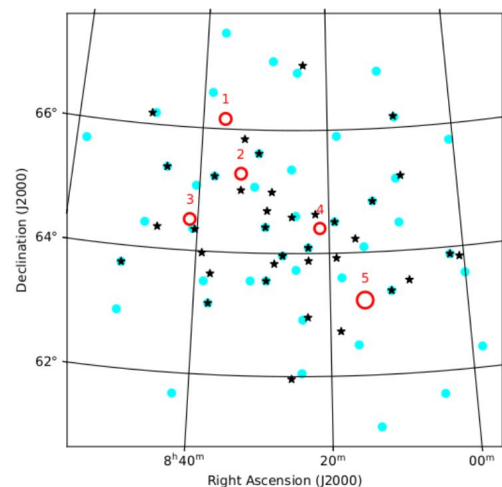
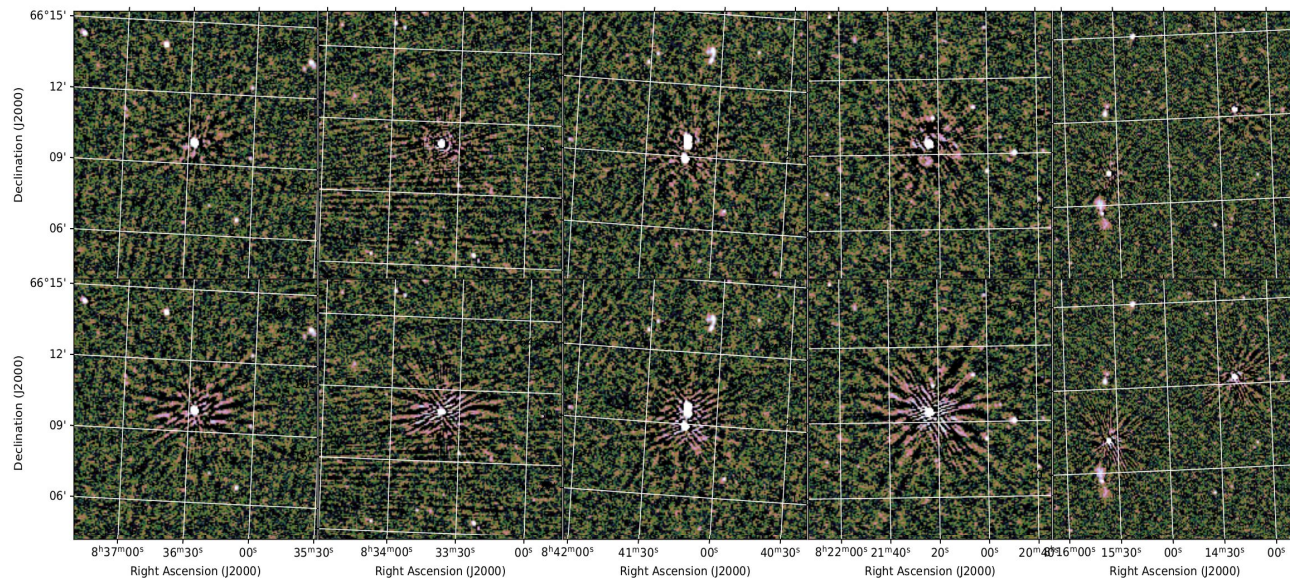
Competitor GP models

Also works in sparse+noisy situations like this



Applying the tomographic kernel to real data

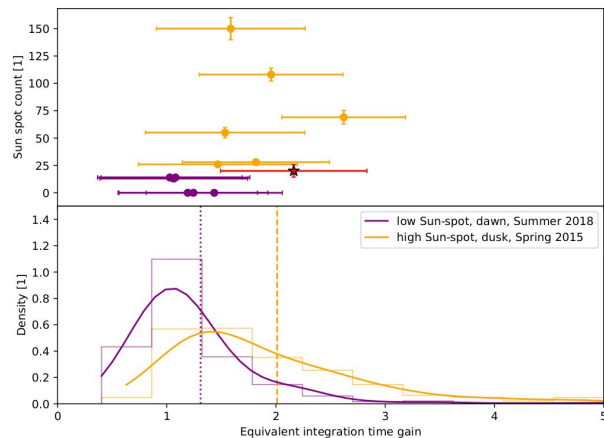
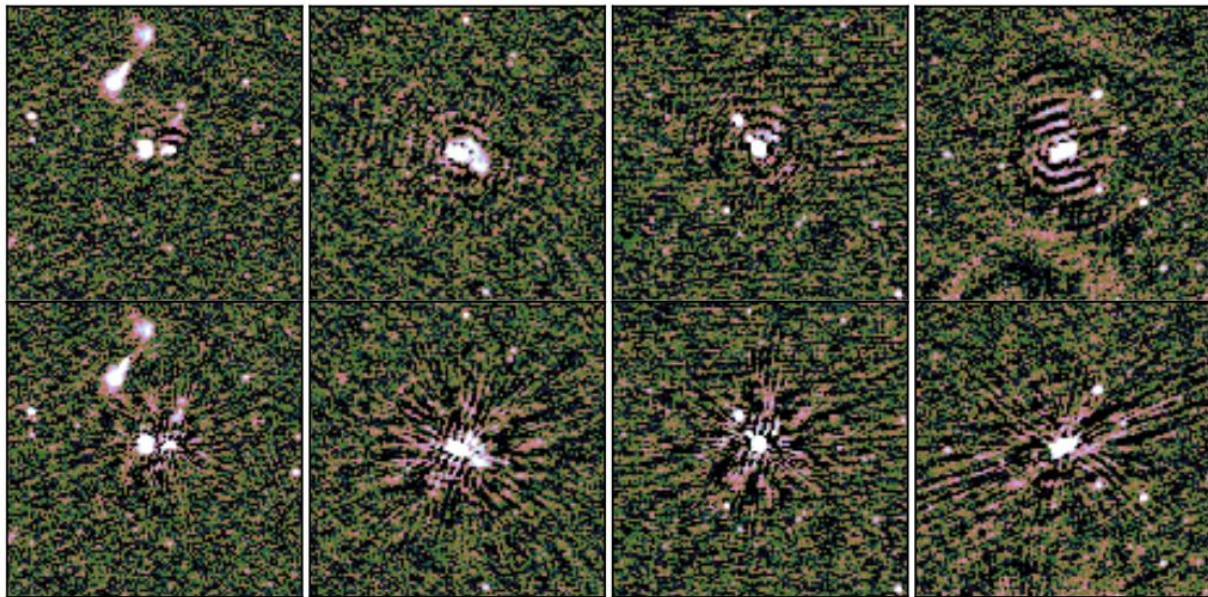
Equivalent integration time gain approx. 2 around inter-calibrator sources



Randomly selected 8 hour LOFAR-HBA observation

Applying the tomographic kernel to real data

Avg. equivalent integration time gain over 12 observations: 1.75

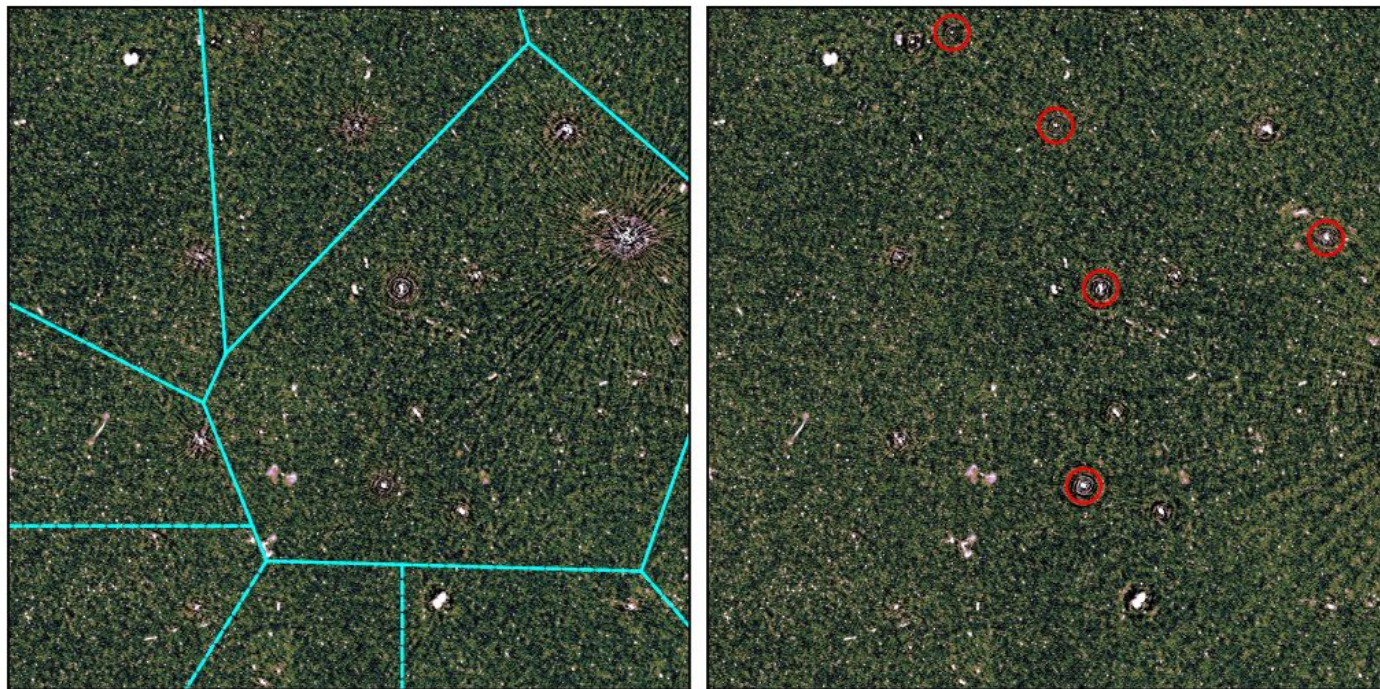


Improvement depends on how strong the Sun is that day, and time of day.

Variable improvements when applied to many observations

Applying the tomographic kernel to 100 hours

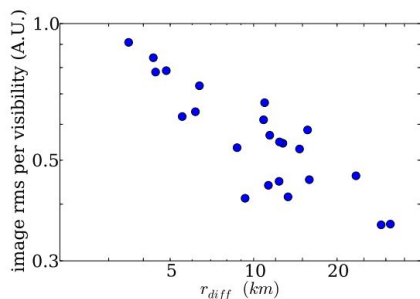
Deep Lockman Hole: 25 μ Jy/beam at 150MHz uniformly across the FoV



Wealth of information on the ionosphere

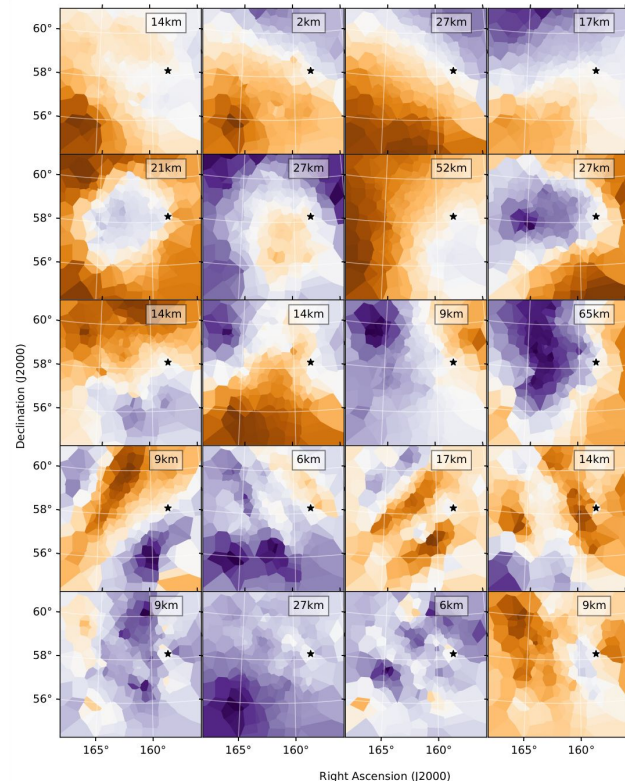
Can we understand the ionosphere better to benefit calibration?

Anyone interested in working on the data or method is welcome to contact me.



M. Meivius et al 2016

We can now go beyond structure functions and talk about discrete behaviours.



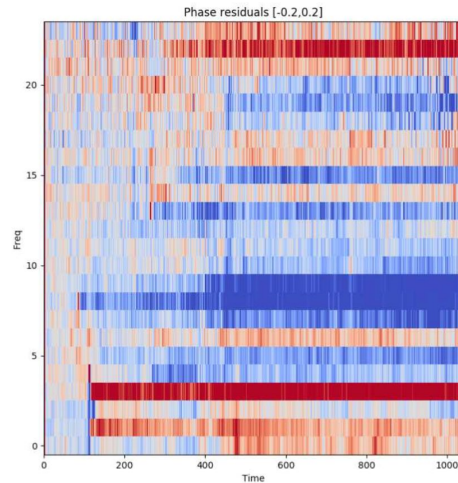
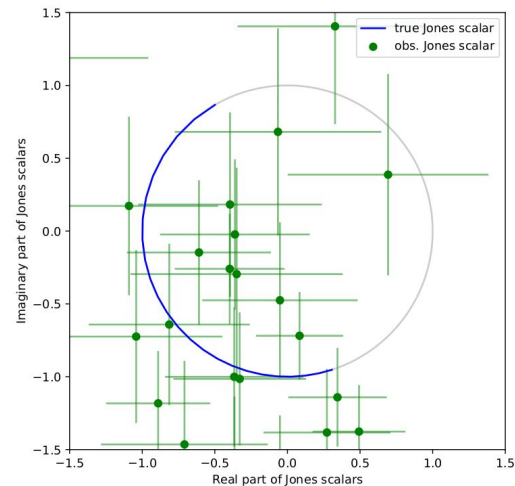
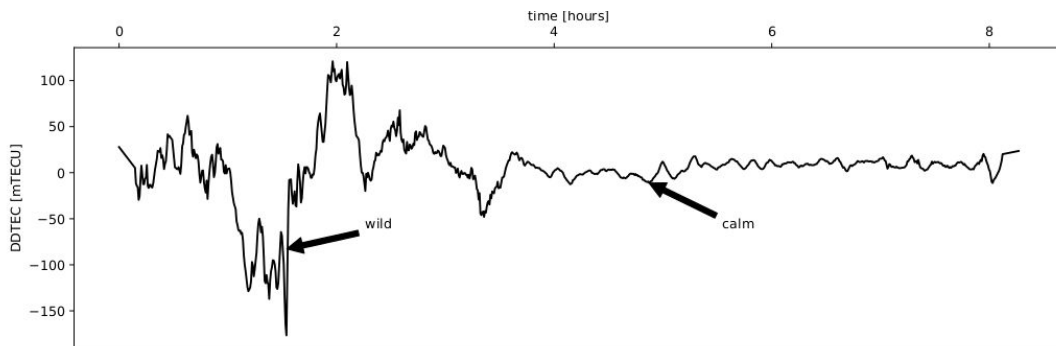
The challenge of working on gains

Modelling uncertainty

Phase wrapping with outliers

Some directions/channels have noisy gains

Ionosphere characteristics change over observation



The challenge of working on gains: outliers

The main challenge is robustness to outliers. Even with neural network based outlier detection false-negative-rate is still too high.

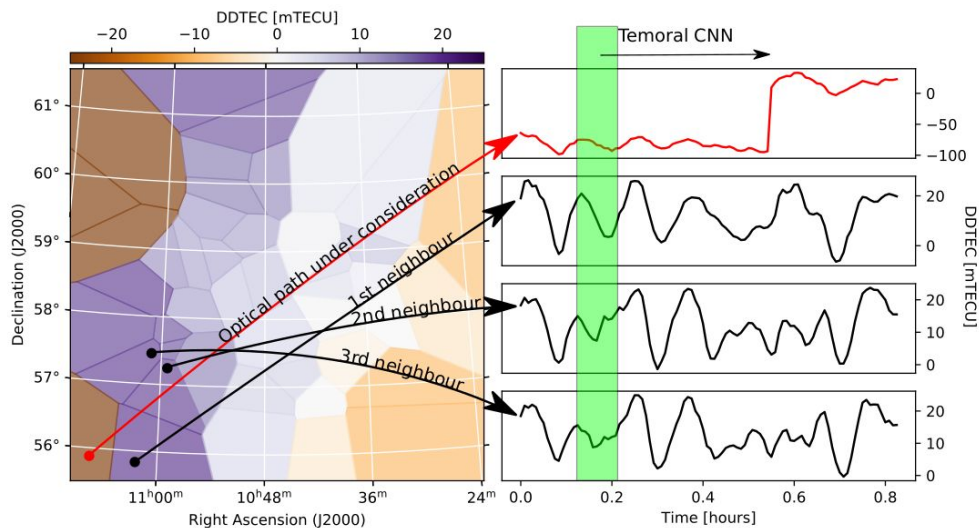


Table 4.2: Comparison of outlier detection methods.

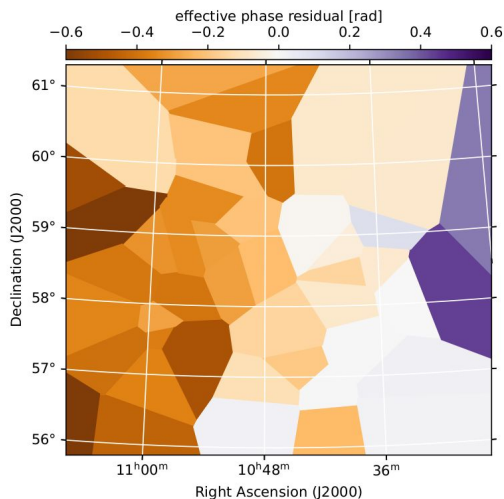
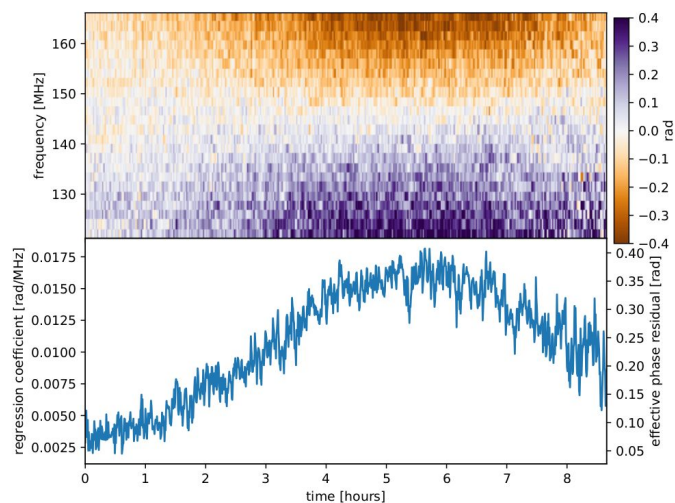
Obs. ID	Neural network method				Heuristic method			
	FNR	FPR	Est. FN	Est. FP	FNR	FPR	Est. FN	Est. FP
667218	1.5%	11.2%	728	274 474	45.2%	5.0%	21 999	122 109
667204	0.4%	4.4%	125	113 679	54.8%	6.1%	19 387	155 669
664480	0.1%	10.4%	49	275 810	58.7%	11.4%	22 768	300 110
664320	1.4%	18.3%	1 500	461 773	49.1%	8.2%	51 741	206 502
659948	1.5%	6.0%	1 247	150 594	45.8%	5.1%	38 239	128 304
659554	0.6%	7.5%	452	195 082	39.3%	7.2%	28 220	187 883
342938	2.2%	11.2%	1 056	320 300	42.3%	5.7%	20 422	161 647
340794	5.9%	5.4%	6 148	152 023	43.7%	4.2%	45 270	118 631
299961	2.1%	2.8%	806	76 993	43.6%	3.2%	17 068	88 911
294287	12.4%	6.9%	25 327	185 568	43.7%	6.9%	89 487	186 491
281008	1.6%	4.2%	532	119 309	42.4%	3.7%	13 705	107 378
274099	4.7%	9.9%	17 329	250 393	48.8%	5.9%	181 955	150 008
Total	2.6%	7.0%	55 298	2 575 997	48.0%	5.5%	550 261	1 913 642

Note: On average there are 2 749 800 optical paths per observation.

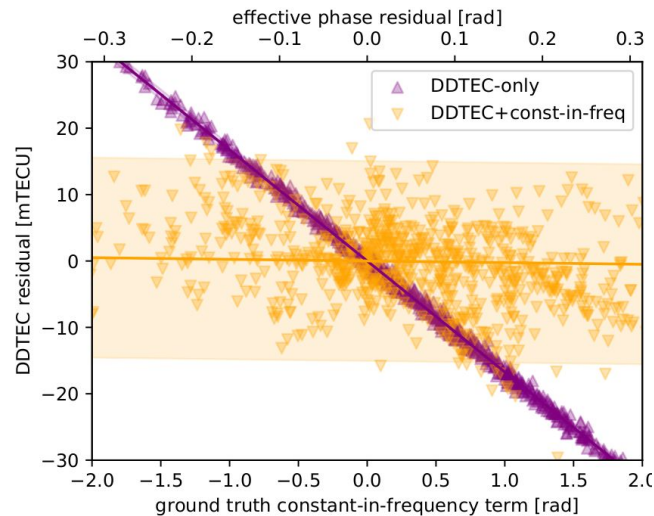
Note: FNR is false-negative rate, FPR is false-positive rate, FN is false negatives, and FP is false positives.

The challenge of working on gains: systematics

The second challenge is modelling non-ionospheric systematics. E.g. a slowly evolving DD phase-component.



High-bias $\phi = \nu^{-1} \tau$
High-variance $\phi = \nu^{-1} \tau + \phi_0$



Review: Recursive Bayesian Estimation (RBE)

Related problems:

- optimal control theory,
- reinforcement learning (POMDP),
- linear dynamical systems,
- Kalman/Rauch/Benes systems,
- quantum mechanics,
- Feynman-Kac particle filters,
- The Scientific Method

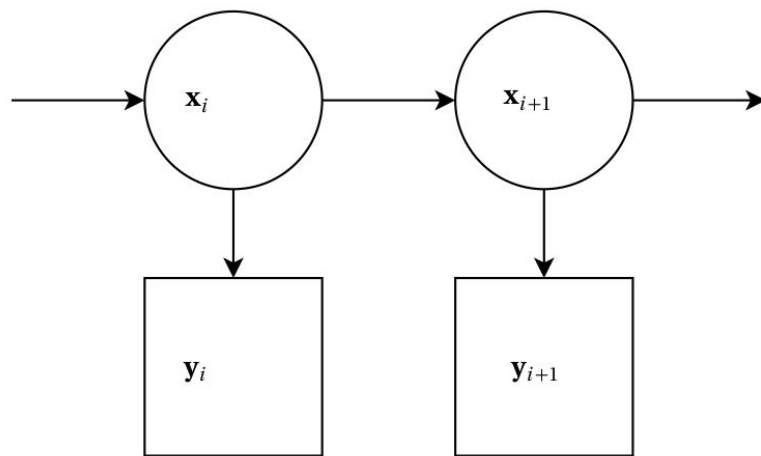


Figure D.1: Casual graph depicting a hidden Markov model.

Review RBE: schematic

Transfer distribution.

Hidden state.

Likelihood.

Observable.

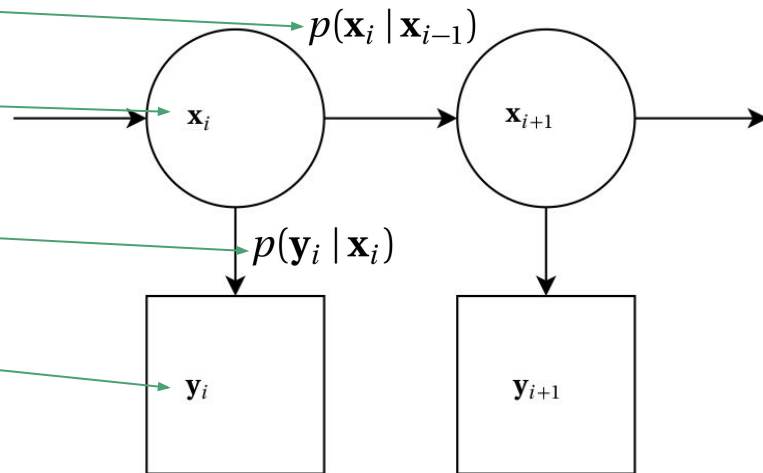


Figure D.1: Casual graph depicting a hidden Markov model.

Review RBE: the transfer distribution

Many possible forms. Questions to ask yourself:

1. Is your hidden state physically interpretable?
2. Is there a known deterministic component in the change of the hidden state?
3. Are there unmodelled components affecting the change in your hidden state?
4. Is your hidden state a martingale (Zero expected change)?
5. Are there constraints on allowed values of the hidden state?

$$\mathcal{U}[\mathbf{x}_i - \boldsymbol{\beta}, \mathbf{x}_i + \boldsymbol{\beta}]$$

$$p(\mathbf{x}_i \mid \mathbf{x}_{i-1}) \quad ??$$

$$\mathcal{N}[\mathbf{x}_i, \boldsymbol{\Sigma}]$$

$$\mathcal{N} \left[\int_{t_i}^{t_{i+1}} f(\mathbf{x}(t'), t') dt', \boldsymbol{\Sigma} \right]$$

Review RBE: the likelihood

Represents your observables, as well as the observation process. Ask yourself,

1. Is there a physical relation between your hidden state and the observable?
2. Does your observation process have non-additive errors?
3. Does your observation process have a prior known uncertainty?
4. Is your modelling uncertainty comparable to your observation errors?
5. Are your observables homogeneous in time (do you have contaminated data)?

$$p(\mathbf{y}_i \mid \mathbf{x}_i) \quad ?? \quad \begin{matrix} \mathcal{N}[f(\mathbf{x}_i), \Sigma_i] & \mathcal{N}_{\mathbb{C}}[f(\mathbf{x}_i), g(\mathbf{x}_i)] \\ \mathcal{L}[f(\mathbf{x}_i), g(\mathbf{x}_i)] \end{matrix}$$

Review RBE: Markov state assumption

The HMM structure imposes conditional relations,

$$P(x_i \mid x_{i-1}, \dots, x_0) = P(x_i \mid x_{i-1})$$

$$P(y_i \mid x_i, \dots, x_0, y_{i-1}, \dots, y_0) = P(y_i \mid x_i)$$

So, the joint distribution of the hidden state and observables is,

$$p(\mathbf{x}_{0:T}, \mathbf{y}_{0:T}) = p(\mathbf{x}_0) \prod_{i=1}^T p(\mathbf{x}_i \mid \mathbf{x}_{i-1}) p(\mathbf{y}_i \mid \mathbf{x}_i).$$

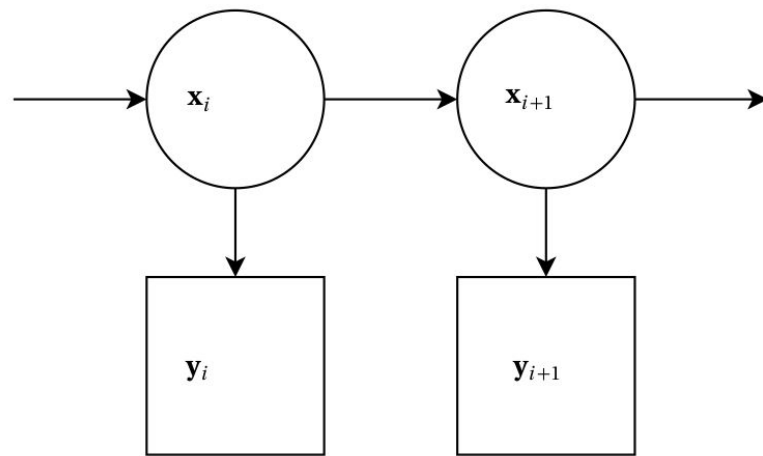


Figure D.1: Casual graph depicting a hidden Markov model.

Review RBE: forward equation

Predict step: what do we expect the state to be at the next step?

$$p(\mathbf{x}_{i+1} \mid \mathbf{y}_{0:i}) = \mathbb{E}_{\mathbf{x}_i \mid \mathbf{y}_{0:i}} [p(\mathbf{x}_{i+1} \mid \mathbf{x}_i)]$$

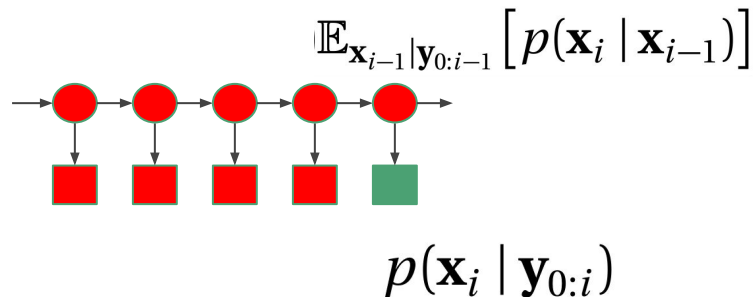
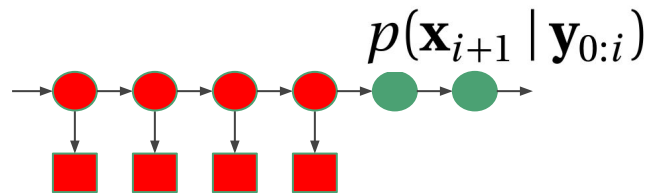
- This becomes our next prior.

Update step: condition the prior on data.

$$p(\mathbf{x}_i \mid \mathbf{y}_{0:i}) = \frac{p(\mathbf{y}_i \mid \mathbf{x}_i) \mathbb{E}_{\mathbf{x}_{i-1} \mid \mathbf{y}_{0:i-1}} [p(\mathbf{x}_i \mid \mathbf{x}_{i-1})]}{p(\mathbf{y}_i \mid \mathbf{y}_{0:i-1})}$$

- Collectively called “forward equation”.

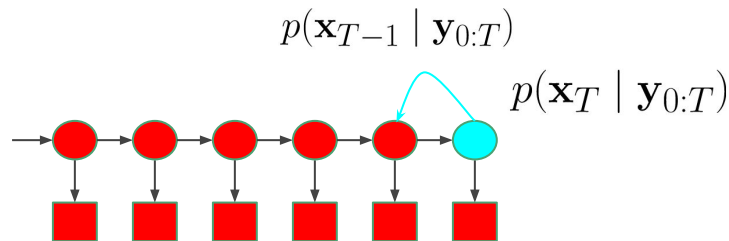
Our belief in the future hidden state is the expectation of the transition distribution.



Review RBE: backward equation

What is the distribution of the hidden state conditioned on *all* the data?

$$p(\mathbf{x}_i | \mathbf{y}_{0:T}) = p(\mathbf{x}_i | \mathbf{y}_{0:i}) \int \frac{p(\mathbf{x}_{i+1} | \mathbf{x}_i) p(\mathbf{x}_{i+1} | \mathbf{y}_{0:T})}{p(\mathbf{x}_{i+1} | \mathbf{y}_{0:i})} d\mathbf{x}_{i+1}$$



- This is called the “backward equation”
- No new information comes from observations, only from back-projecting the posterior $\Rightarrow$ You don’t need to keep the data.
- The backward pass is an improvement that “comes for free”.

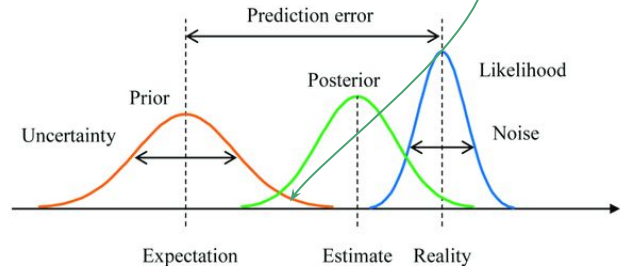
Information gain

How much information is gained by updating the prior with a new observation?

$$IG = -H = D_{\text{KL}}(P(\theta|Y) || P(\theta))$$

Overlapping prior volume $\approx e^H$

- Observations provide zero information $\Leftrightarrow$ prior is the posterior.
- **Number of computations scale with IG**
 - We can use this to our advantage!



Information gain in RBE

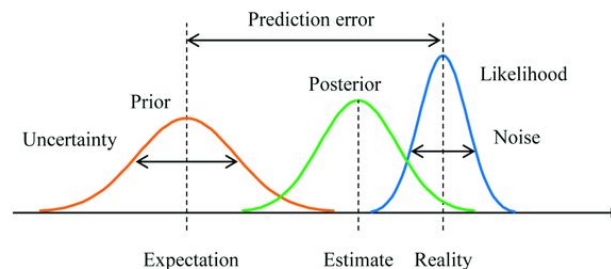
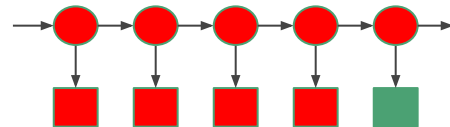
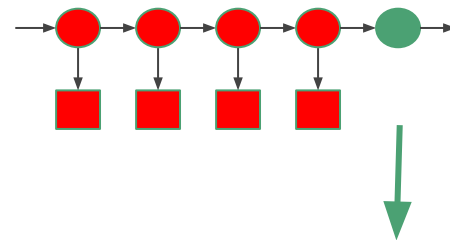
In RBE, we have good reason to believe the prior is close to the posterior.

$$IG_{i+1} = D_{\text{KL}}(P(\theta_{i+1}|Y_{0:i+1}) || \mathbb{E}_{\theta_i|Y_{0:i}}[P(\theta_{i+1}|\theta_{i+1})])$$

The information gain of each filtering update tends to be small.

$$P(\theta_{i+1}|Y_{0:i+1}) \approx P(\theta_{i+1}|Y_{0:i}) \implies IG_{i+1} \rightarrow 0$$

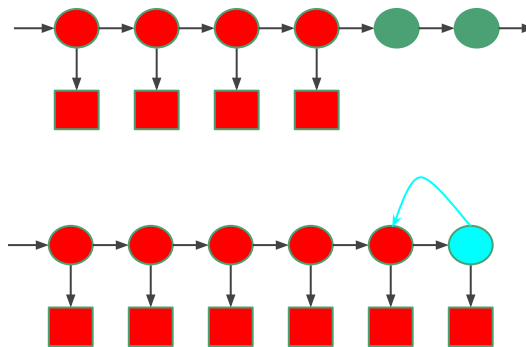
The number of computations required to perform inference drastically decrease!



Small tangent: The Scientific Method

An interesting philosophical point. Which of the following two claims is correct?

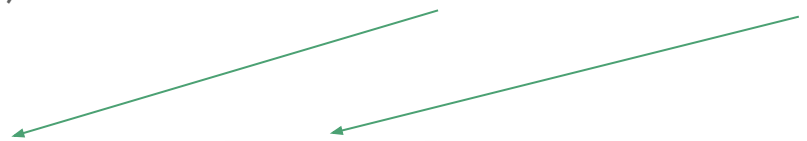
1. The Scientific Method only needs the forward equation, i.e. it's a filtering problem.
2. The Scientific Method necessarily must use both the forward and backward equations, i.e. it's a smoothing problem.



It's a smoothing problem. At some point we will exhaust our observations, gaining no new information from any new observation $IG = 0$. All science will depend upon reanalysing our past priors with all the data gathered since $IG \neq 0$ for the smoothing problem.

RBE: analytically tractable Gaussianity

Kalman (1960) and Rauch (1963) deal with Gaussian **likelihood** and **transfer distribution**.


$$p(\mathbf{x}_i | \mathbf{y}_{0:i}) = \frac{p(\mathbf{y}_i | \mathbf{x}_i) \mathbb{E}_{\mathbf{x}_{i-1} | \mathbf{y}_{0:i-1}} [p(\mathbf{x}_i | \mathbf{x}_{i-1})]}{p(\mathbf{y}_i | \mathbf{y}_{0:i-1})}$$

The forward equations : Kalman equations

The backward equations: Rauch equations.

Non-Gaussian likelihoods: unscented, extended, particle filter, Benes filter.

Used by KillMS calibration (C. Tasse 2014b)

RBE: non-Gaussianity with variational inference

What about when the likelihood is not self-conjugate? E.g. $\mathcal{N}_{\mathbb{C}}[f(\mathbf{x}_i), g(\mathbf{x}_i)]$

$$p(\mathbf{x}_i | \mathbf{y}_{0:i}) = \frac{p(\mathbf{y}_i | \mathbf{x}_i) \mathbb{E}_{\mathbf{x}_{i-1} | \mathbf{y}_{0:i-1}} [p(\mathbf{x}_i | \mathbf{x}_{i-1})]}{p(\mathbf{y}_i | \mathbf{y}_{0:i-1})}$$

Approximate the posterior (LHS) is a Gaussian distribution, then **how do you choose the parameters that best match the true posterior?**

$$q(x_i | y_{0:i}) = \mathcal{N}[\mu, \Gamma]$$

$$m, \Gamma = \arg \min KL(q || p)$$

Turns a sampling problem into an optimisation problem! Nice!

RBE: fully Bayesian forward equation

What about when the likelihood is not self-conjugate?

$$p(\mathbf{x}_i | \mathbf{y}_{0:i}) = \frac{p(\mathbf{y}_i | \mathbf{x}_i) \mathbb{E}_{\mathbf{x}_{i-1} | \mathbf{y}_{0:i-1}} [p(\mathbf{x}_i | \mathbf{x}_{i-1})]}{p(\mathbf{y}_i | \mathbf{y}_{0:i-1})}$$



Set prior to $\mathbb{E}_{\mathbf{x}_{i-1} | \mathbf{y}_{0:i-1}} [p(\mathbf{x}_i | \mathbf{x}_{i-1})]$

Set the likelihood to $p(\mathbf{y}_i | \mathbf{x}_i)$

We can exactly compute this conditioning operation with nested sampling.

JAX powered non-linear dynamical systems

Joshuaalbert / jaxnlds

<> Code ⓘ Issues 🔍 Pull requests ⌚ Actions 📁 Projects 📖 Wiki 🛡 Security 📊 Insights ⚙ Settings

🔗 master 🔗 1 branch 🔗 0 tags Go to file Add file ↕ Code ↕

Joshuaalbert Merge remote-tracking branch 'origin/master' 3c4a252 · on Aug 21 🕒 11 commits

📁 jaxnlds	* hmm with nested sampling example	2 months ago
📄 .gitignore	Initial commit	2 months ago
📄 LICENSE	Initial commit	2 months ago
📄 README.md	Update README.md	2 months ago
📄 install_requirements.sh	* add missed files	2 months ago
📄 requirements.txt	* remove unused deps	2 months ago
📄 setup.py	Update setup.py	2 months ago

README.md

jaxnlds

Inference on non-linear dynamical systems written in JAX.

Install

Requires python 3.X.

As always I recommend a package manager like `conda`. In this case you could do `conda create -n jaxnlds_py python=3.8 && conda activate jaxnlds_py`

Exact and approximate solutions to RBE problems with both Kalman/Rauch, variational inference, and nested sampling.

Fully JIT compiles to CPU, GPU, or TPU.

[1] <https://github.com/Joshuaalbert/jaxnlds>



RBE: variational inference (on gains)

Phase model $\phi = \nu^{-1} \tau$

TEC prior $\tau \sim \mathcal{N}(\mu, \sigma^2)$

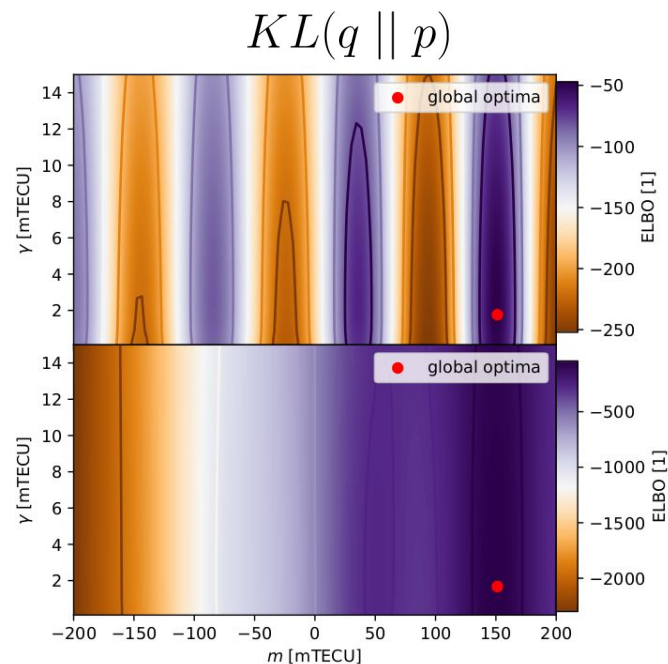
Variational expectation = $\mathbb{E}_{\tau}[\mathcal{N}_{\mathbb{C}}(\mathbf{g} | g e^{i\nu^{-1}\tau}, \sigma^2)]$

Population prior

RBE prior

Result: The recursive prior solves the phase wrapping problem.

With JAXNLDS, solves approximately 2000 gains/second on CPU.

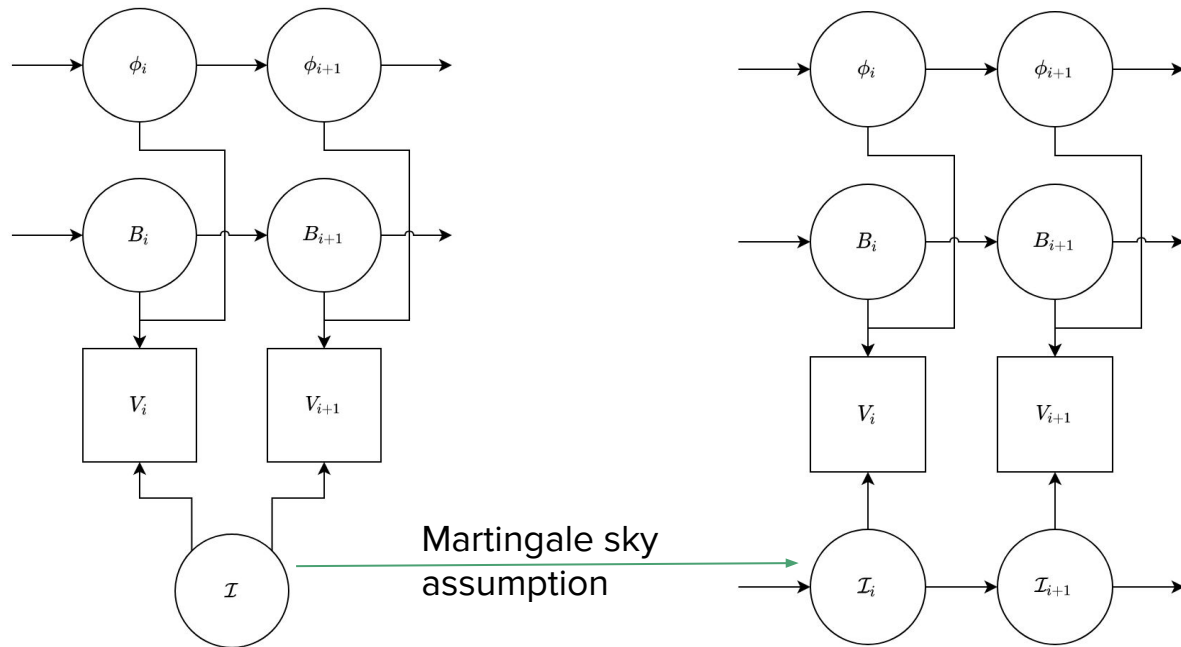


RBE: applying it to broader class of problems

When formulating an RBE, try to build hierarchical models with Martingale priors.

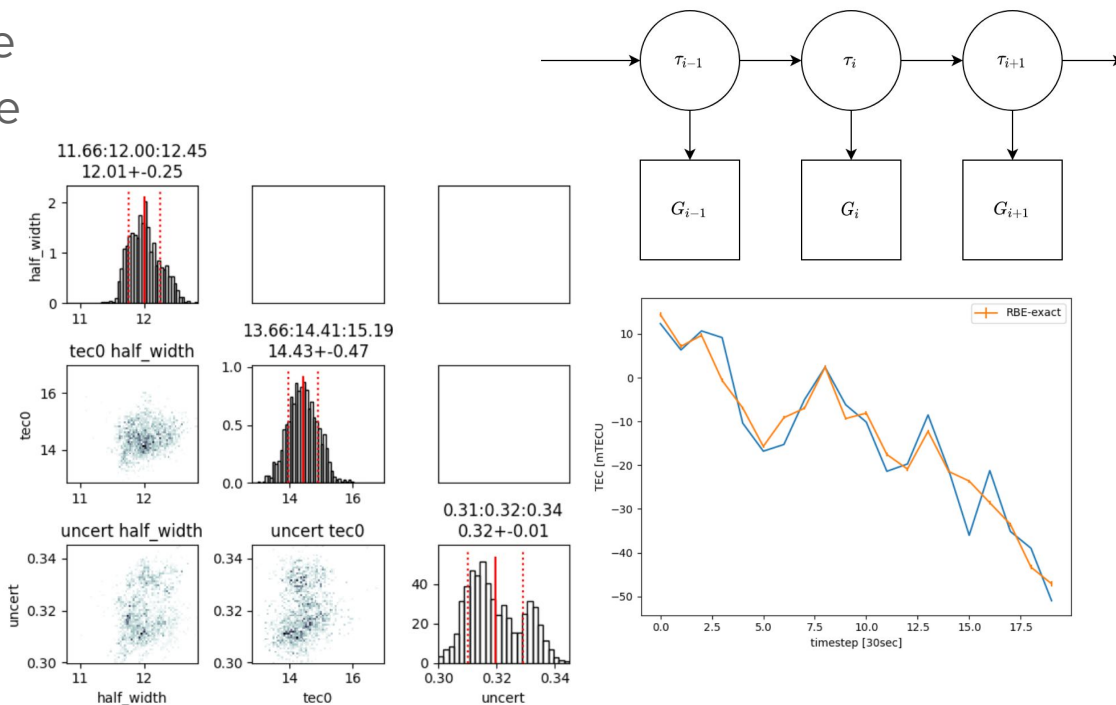
The less the prior changes
 $\Rightarrow$ the smaller the information gain
 $\Rightarrow$ fewer computations to solve

To use RBE-VI, all we need is,
 $\text{VarExp} = \mathbb{E}_{\phi_i, B_i, \mathcal{I}_i}[V_i]$



Example: Using RBE to infer total electron content

By setting a strong prior for the first point, IG and compute time decrease by 20%.



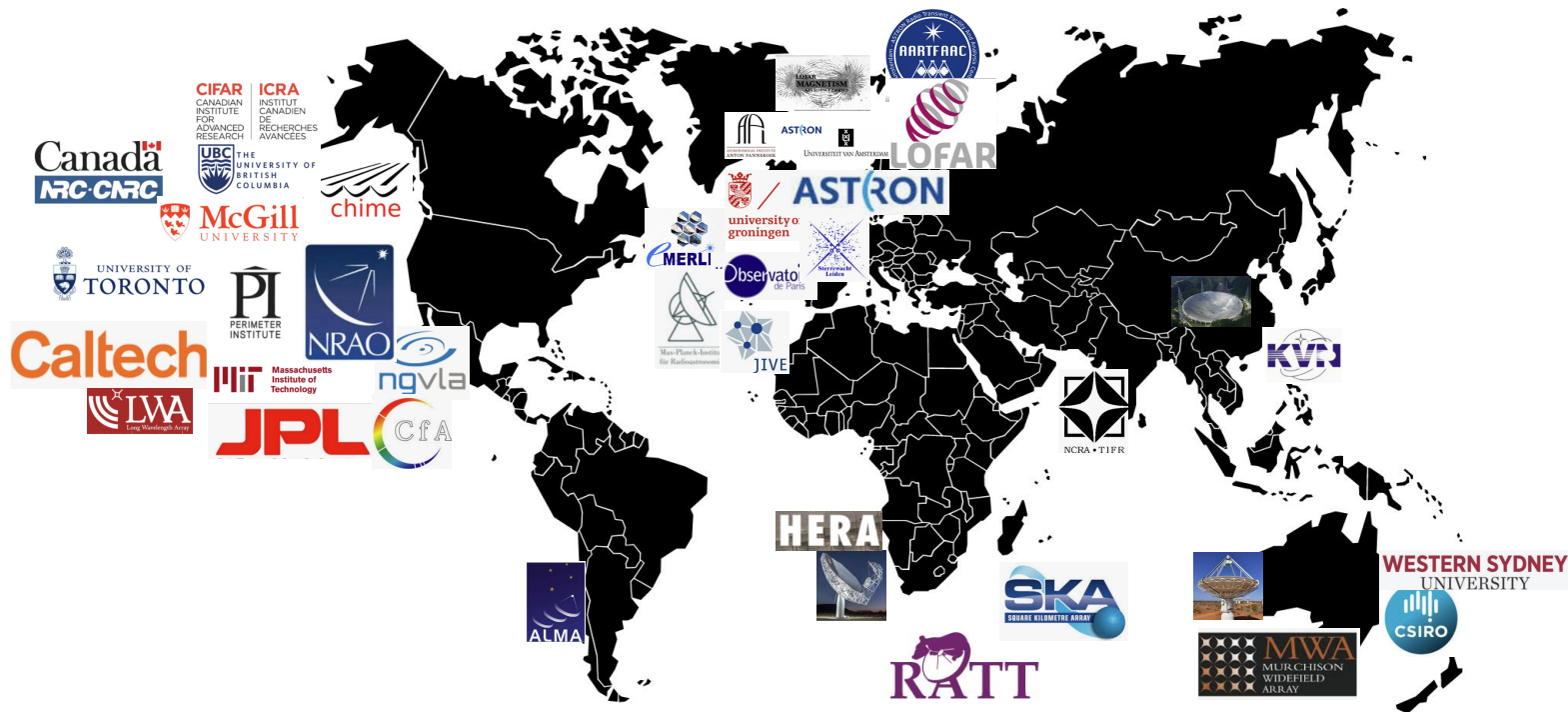
Radio Interferometry Kaggle (RIK)

*Likely to be merged with RCI
Image Contest*

The objectives of Radio Interferometry Kaggle are:

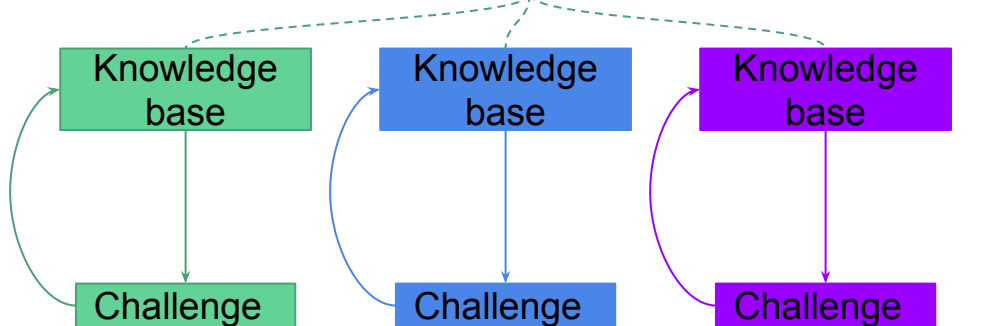
1. To eliminate technique and pipeline (TP) "camping".
2. To provide a standardised comparison of TPs.
3. To provide an easily accessible arsenal of state-of-art TPs to everyone.
4. To provide unified distributed processing of TPs across an industry-subsidised massive compute resource network.
5. To enable maximal efficiency of getting new ideas into a pipeline, tested, and into production.
6. To give novice radio astronomers interesting and challenging problems to contribute and practice their skills.

Radio Interferometry Kaggle



RIK: currently “competition without the benefits”

Slow trickle of science between camps

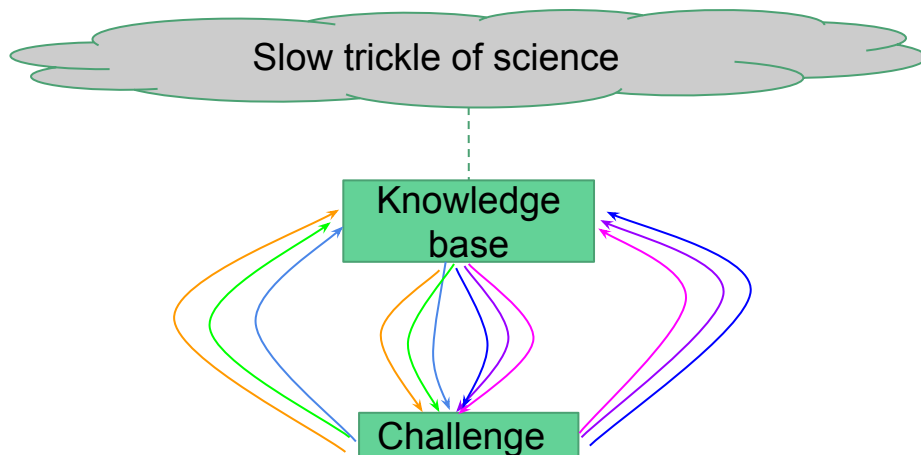


People get grants to develop methods.



1. The challenges are self-imposed and different, so cross comparison is difficult.
2. The knowledge bases are partially disjoint, so optimisation is somewhat redundant.
3. Politics dominate and not the intrinsic value of the method.
4. Everyone uses different frameworks, so it becomes very difficult to share methods even if you wanted to.

RIK: more beneficial competitive optimisation



People *still* get grants to develop methods.



1. The challenges are the identical, so cross comparison is easy and well-defined.
2. The knowledge bases are identical, so optimisation is building on the state-of-the-art methods.
3. Intrinsic value of the method is dominates.
4. Everyone uses the pipeline framework, so it becomes as easy as dragging and dropping to share methods.

What is Kaggle? *Controlled efficient beneficial competition in data science.*

“Kaggle is an AirBnB for Data Scientists – this is where they spend their free nights and weekends.” With 5 million users only 196 Kaggle Grandmasters to date.

- Solved a multitude of big problems ranging from HIV research to anti-terrorism.
- Interesting and challenging projects where contributors can learn and practice.
- Insightful discussions with industry leaders.

In short:

- **“*The most valuable resource if you’re a data scientist*”**



196
Grandmasters



1,476
Masters



6,026
Experts



56,843
Contributors



83,101
Novices

Rank	Tier	User	Medals	Points
1		 bestfitting joined 4 years ago	 28  8  1	221,687
2		 Guanshuo Xu joined 5 years ago	 15  15  2	204,592
3		 Psi joined 9 years ago	 13  5  0	192,493
4		 Μαριος Μιχαηλιδης KazAnova joined 7 years ago	 38  52  38	169,473



What is Kaggle? *controlled efficient beneficial competition in data science.*

Home

Compete

Data

Notebooks

Discuss

Courses

Jobs

More

Recently Viewed

Conway's Reverse Ga...

RSNA STR Pulmonary ...

I'm Something of a Pai...

Mechanisms of Action ...

Tutorial Notebook

Search

Competitions

Grow your data science skills by competing in our exciting competitions. Find help in the [Documentation](#) or learn about [InClass competitions](#).

Your Competitions

Active Closed Pinned Hosted



Contradictory, My Dear Watson
Detecting contradiction and entailment in multilingual text using TPUs
Getting Started • Ongoing • Code Competition • 243 Teams



RSNA STR Pulmonary Embolism Detection
Classify Pulmonary Embolism cases in chest CT scans
Featured • 16 days to go • Code Competition • 493 Teams



Lyft Motion Prediction for Autonomous Vehicles
Build motion prediction models for self-driving vehicles
Featured • a month to go • Code Competition • 577 Teams



Mechanisms of Action (MoA) Prediction
Can you improve the algorithm that classifies drugs based on their biological activity?
Research • 2 months to go • Code Competition • 2375 Teams



Google Research Football with Manchester City F.C.
Train agents to master the world's most popular sport
Featured • 2 months to go • Simulation Competition • 595 Teams



Conway's Reverse Game of Life 2020
Reverse the arrow of time in the Game of Life
Playground • 2 months to go • Code Competition • 101 Teams



Hash Code Archive: Drone Delivery
Can you help coordinate the drone delivery supply chain?
Playground • 3 months to go • Code Competition • 60 Teams

All Competitions

Active (Not Entered) Completed InClass

Riiid! Answer Correctness Prediction
Track knowledge states of 1M+ students in the wild
Featured • 3 months to go • Code Competition • 400 Teams

RSNA STR Pulmonary Embolism Detection
Classify Pulmonary Embolism cases in chest CT scans
Featured • 16 days to go • Code Competition • 493 Teams

Lyft Motion Prediction for Autonomous Vehicles
Build motion prediction models for self-driving vehicles
Featured • a month to go • Code Competition • 577 Teams

Mechanisms of Action (MoA) Prediction
Can you improve the algorithm that classifies drugs based on their biological activity?
Research • 2 months to go • Code Competition • 2375 Teams

Google Research Football with Manchester City F.C.
Train agents to master the world's most popular sport
Featured • 2 months to go • Simulation Competition • 595 Teams

Conway's Reverse Game of Life 2020
Reverse the arrow of time in the Game of Life
Playground • 2 months to go • Code Competition • 101 Teams

Hash Code Archive: Drone Delivery
Can you help coordinate the drone delivery supply chain?
Playground • 3 months to go • Code Competition • 60 Teams

\$30,000

\$30,000

\$6,000

Swag

Knowledge

Conway's Reverse Game of Life 2020

Reverse the arrow of time in the Game of Life

Kaggle · 101 teams · 2 months to go

Overview Data Notebooks Discussion Leaderboard Rules

Join Competition

Overview

Description

Evaluation

Timeline

Prizes

Code Requirements

This is a relaunch of a previous competition, [Conway's Reverse Game of Life](#), with the following changes:

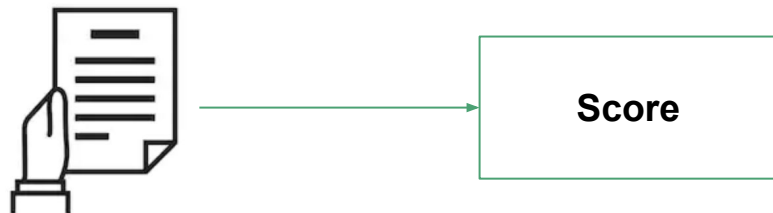
- The grid size is larger (25 vs. 25) and the grid wraps around from top to bottom and left to right
- Submissions are solved forward by the appropriate number of steps, so that any correct starting

#	Team Name	Notebook	Team Members	Score	Entries	Last
1	Vlad Golubev			0.02234	10	1d
2	Robert C			0.02296	17	2d
3	Kota Iizuka			0.02668	5	3d
4	Yuri Matiounine			0.03361	13	10d
5	cj			0.03556	12	6d
6	Mark #2			0.03750	6	5d
7	rc95401			0.04585	21	7h
8	Deeper Systems			0.04596	11	2d

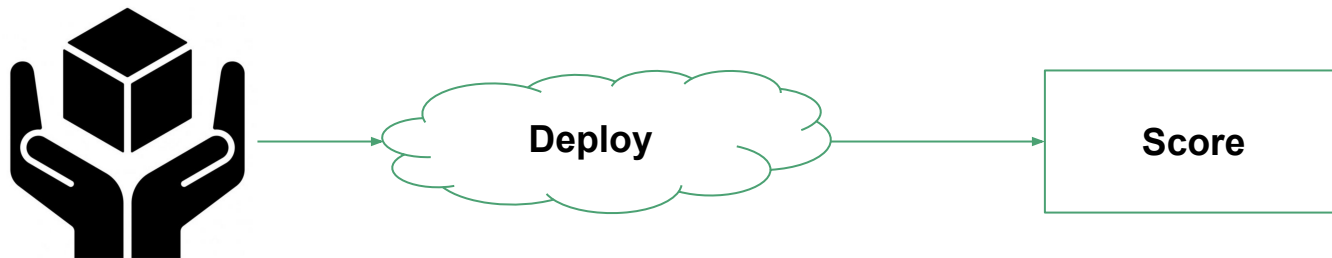
★★★★★

How RIK's different from Kaggle

In Kaggle you submit results and the score method runs on this submission.



In RIK you submit a pipeline, which is deployed for you. The results are collected from this submission, and computational resource usage is automatically counted.



What RIK is and isn't

What it is:

- A concept much like Kaggle.
- A UI/API where all previously registered open-sourced pipelines are available.
- A pipeline orchestrator.
- A place for any level radio astronomer to grow and contribute.
- Incentivised with small cash prizes.

What it isn't:

- A camp exclusive tool.
- A deathmatch, e.g. Planck Challenge.
- A new concept.

[1] <https://www.astron.nl/citt/genericpipeline/>

[2] <https://github.com/gijzelaerr/kliko>

[3] <https://www.commonwl.org/>

[4] <https://github.com/gijzelaerr/buis>

[5] <https://github.com/ratt-ru/Stimela>

[6] <https://github.com/caracal-pipeline/caracal>

Key components

1. Creating the challenges
2. Incentivising participation
 - a. Kaggle is proof of “build it and they will come”
 - b. Monetary rewards (Eric Schmidt & Stuart Feldman)
3. Compute resource infrastructure
 - a. Industry and institutional partners
 - b. The ability to provide contestants with the correct hardware for their TP.
4. Implementation
 - a. A system to make all this happen.

Challenges:

- The most rigorous and grand simulations ever.
 - a. Open-source, community driven, and available on RIK.
- Sub-category challenges, e.g.
 - a. Source detection
 - b. Transient detection
 - c. Calibration only
- Meaningful metrics
 - a. Fidelity
 - b. Compute cost
 - c. Uncertainty estimation

Proof of concept implementation

- Originally for data science in medicine.
- Easily, and flexibly define and share pipelines
- Seamlessly orchestrate pipelines across a compute network
- Features:
 - Intelligently bringing code to data
 - Handle radio astronomy's mutable data container, aka measurement set
 - Run on cloud or user's registered compute resources

Touch Intel - Data Science Platform

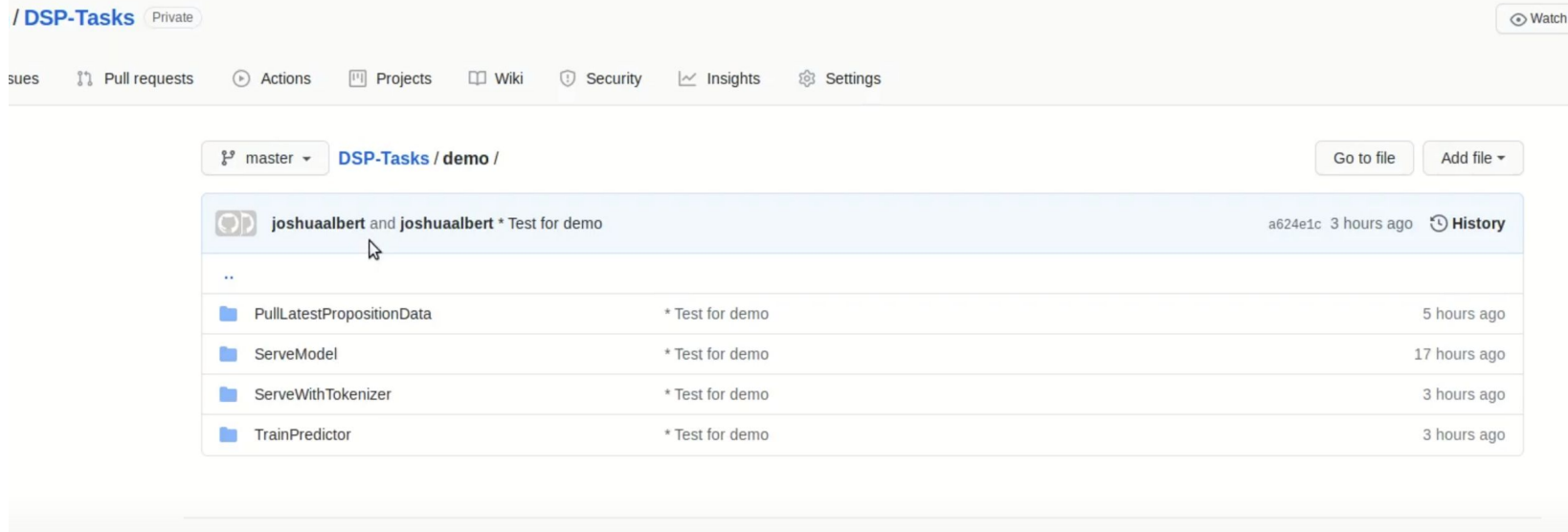


 Login with Github

Tasks are accessible via github

A pipeline consists of tasks distributed across many repos

The task versioning is linked to commits for traceability



The screenshot shows the GitHub interface for the `DSP-Tasks` repository. The repository is marked as `Private`. The navigation bar includes links for `sues`, `Pull requests`, `Actions`, `Projects`, `Wiki`, `Security`, `Insights`, and `Settings`. A `Watch` button is also present.

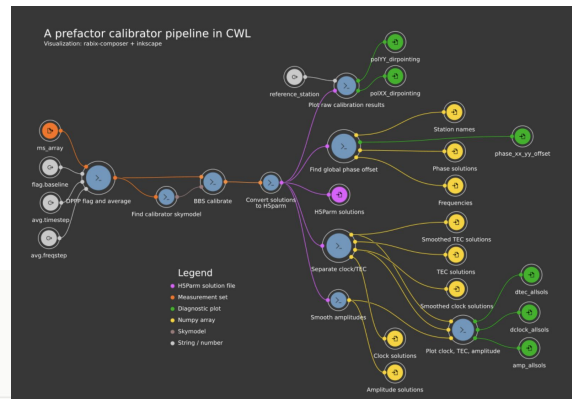
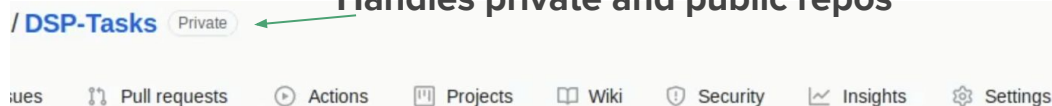
The main content area displays the commit history for the `master` branch. The current commit is by `joshuaalbert` and `joshuaalbert`, titled `* Test for demo`, with commit hash `a624e1c` and made `3 hours ago`. A `History` link is available for this commit.

Below the commit header, a table lists the files changed in this commit:

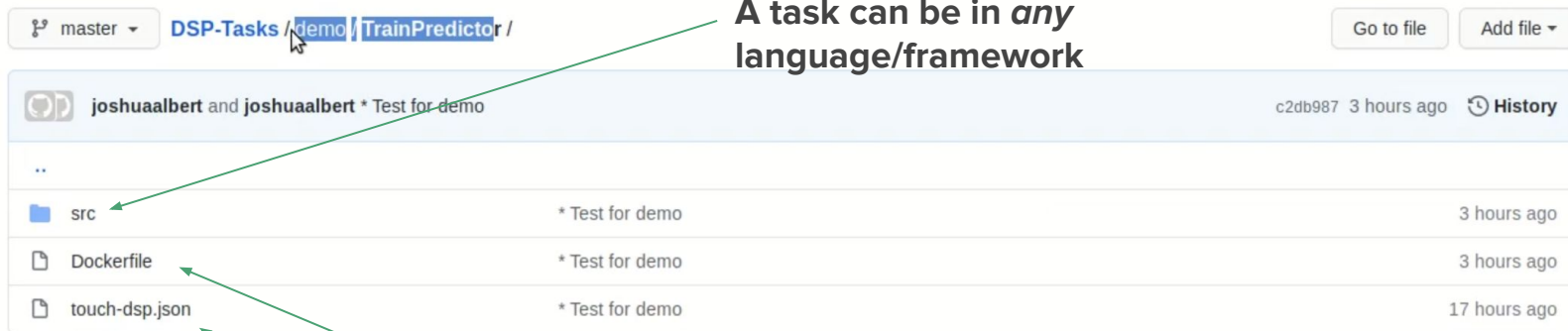
File	Commit Message	Time
..		
<code>PullLatestPropositionData</code>	<code>* Test for demo</code>	5 hours ago
<code>ServeModel</code>	<code>* Test for demo</code>	17 hours ago
<code>ServeWithTokenizer</code>	<code>* Test for demo</code>	3 hours ago
<code>TrainPredictor</code>	<code>* Test for demo</code>	3 hours ago

Tasks are accessible via github

Handles private and public repos



A task can be in *any* language/framework



Define what environment you need per task in a Dockerfile

Define input parameters in a task manifest

Task definitions

Tied to commits

Once defined, anyone can use it to define a task instance

Specify compute group

Git repo

Path to task

Touch Data Science Platform

localhost:3000/task-definitions/new

Apps

TOUCH

Dashboard

Groups

Resources

Compute

Tasks

Task Definitions

Task Instances

Task Graphs

Scheduled Graphs

Runs

Create New Task

* Resource Group:

Touch Internal

* Repository:

https://github.com/Touch-Physio/DSP-Tasks.git

* Branch:

master

* Task Path:

/demo/serveWithTokenizer/

Valid Touch DSP manifest file found.

* Task Name:

+ Create Back

Task instances

Set parameters

Task instances are sharable from other pipelines

The screenshot shows the 'New Task Instance' form in the Touch Data Science Platform. The form is divided into several sections: Task Information, Environment Variables, Attached Networks, and Endpoint Maps. The Task Information section contains fields for Task Name, Task Repository, Task Branch, Task Branch Commit, Task Path, and Task Instance Name. The Environment Variables section has an 'Add Environment Variable' button. The Attached Networks section has an 'Attach Network' button. The Endpoint Maps section has three rows for mapping internal paths to external paths. The form is styled with a dark sidebar on the left and a light gray main area. The 'Create' button is highlighted in blue.

Touch Data Science Platform

localhost:3000/task-instances/new

Apps

TOUCH

Dashboard

Groups

Resources

Compute

Tasks

Task Definitions

Task Instances

Task Graphs

Scheduled Graphs

Runs

New Task Instance

Task Information

* Task Name:

AI-Demo-ServeWithTokenizer

* Task Repository:

https://github.com/Touch-Physio/DSP-Tasks.git

* Task Branch:

master

* Task Branch Commit:

a624e1c9cceede1743bec9227eb028a87bc889bc1

* Task Path:

./demo/ServeWithTokenizer

* Task Instance Name:

AI-Demo-ServeWithTokenizer

Environment Variables

Add Environment Variable

Attached Networks

Attach Network

Endpoint Maps

Task Instance Path:

/http-echo

Map Internal /echo to:

/echo

Map Internal /echo-get to:

/echo-get

Map Internal /another to:

/another

+ Create

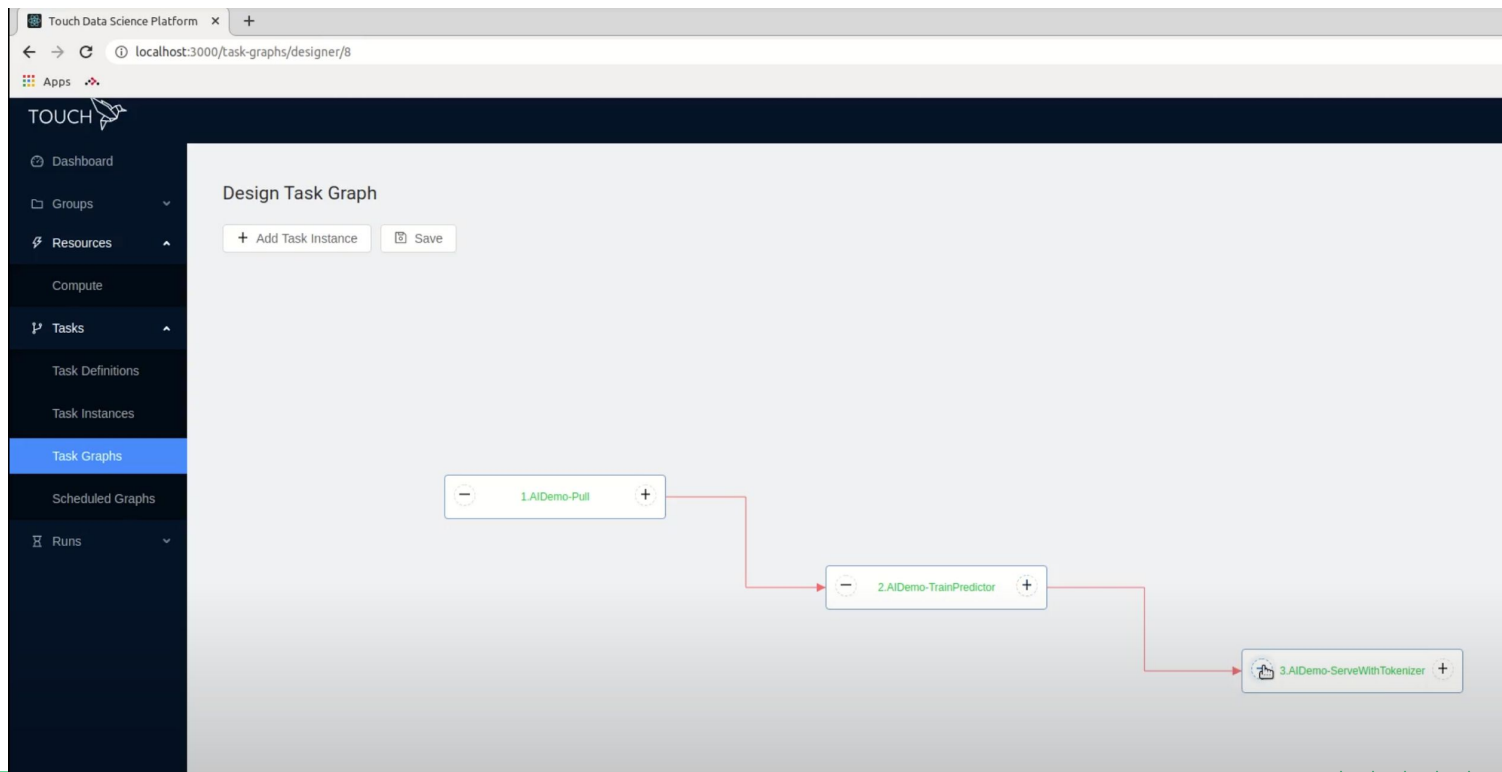
Back

Define the task graph

**Drag-n-drop tasks
and link them into a
DAG**

**Easy to visualise a
task graph**

**Share and modify
pipelines**



Deploy and monitor

Compute resource usage is measured

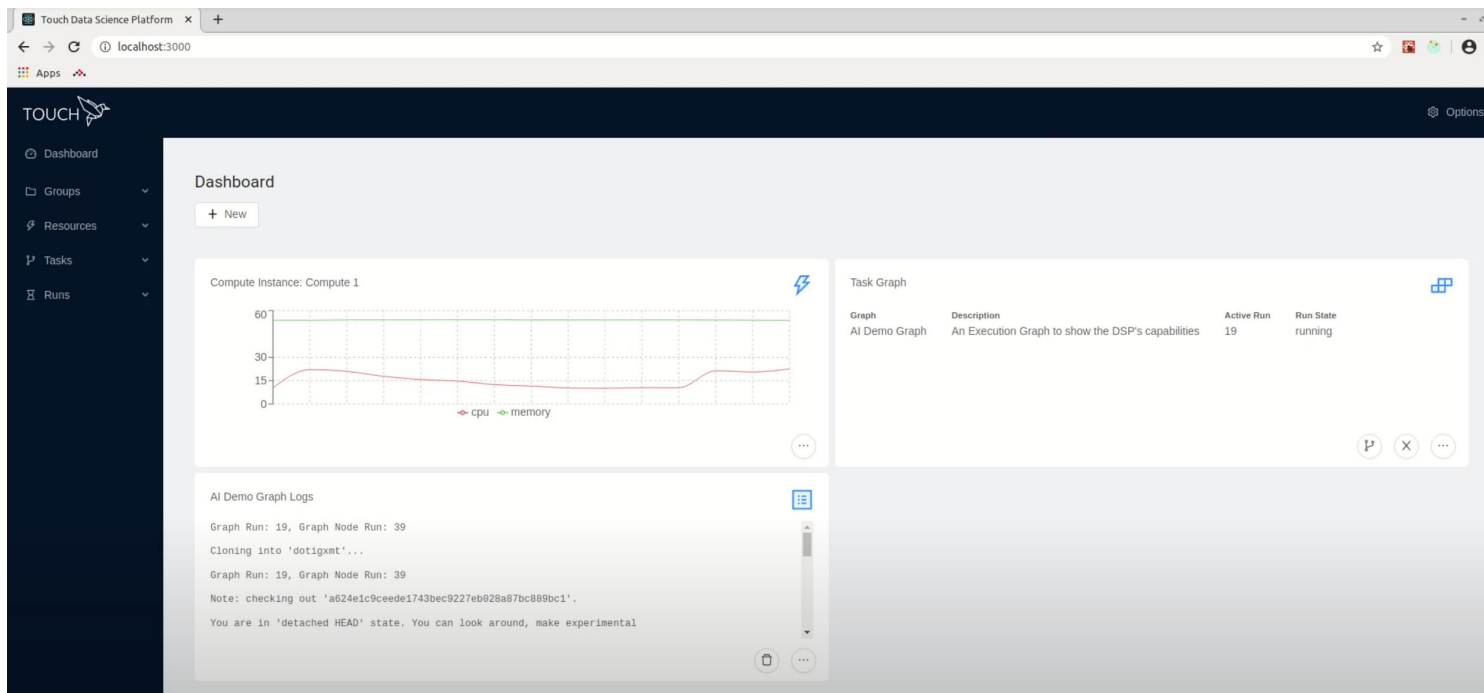
Deployed across compute network

Smart scheduling

Bringing code to data

Handles big-data mutability

VM provider agnostic



RIK: status and upcoming milestones

Status: Small industry partner with data science platform. Gathering interest.

Key milestones:

- End of 2020 - Proof of concept demonstration.
- Spring 2021 - All major radio interferometry camps aware of the project.
- End of 2021 - Acquire key industry partnership and funding for compute resources and prizes.
- End of 2021 - Have initial website live for “registered compute resources”.
 - Webinars and tutorials about how to submit pipelines

Conclusion

1. JAX is an effective way for scientists to write platform-independent HPC code.
2. Born RIME vs Rytov RIME: novel territory to explore.
3. Probabilistic tomographic ionosphere calibration produces good phase screens.
4. Recursive Bayesian estimation: solves difficult sequential problems, and is the formalism for streaming inference.
5. Radio Interferometry Kaggle: Revolutionise and unify radio astronomy calibration and imaging, and science.